Spring Boot Debug REPL and MCP

Részletes felhasználói kézikönyv - 0.24.1

Fülek, gombok, mezők és munkafolyamatok az IntelliJ IDEA pluginhoz. Java-kísérletezés a futó alkalmazásban, objektumböngészés, snapshotok, kódfrissítés és közös munka Claude-dal.

Kinek szól? Ha most kapcsolod be először a REPL-t, kezdd az 1. fejezettel. Ha egy konkrét gomb működését keresed, a tartalomjegyzékből vagy a PDF könyvjelzőiből ugorj a megfelelő fülre. A 24. fejezet a gyorsbillentyűket, a 25. az MCP-eszközöket foglalja össze.

Mit ad a REPL? A futó Java-alkalmazás saját osztályaival és Spring beanjeivel dolgozhatsz. A session megőrzi az importokat, változókat és metódusokat. Egy eredményt többször megvizsgálhatsz, majd célzott DATA snapshotként elmenthetsz.

A kézikönyv alapja: a 0.24.1 forrásában ténylegesen bekötött 12 fő fül, az alfüleik, gombjaik és beállításaik. A gombfeliratokat eredeti formában adjuk meg. Az IDEA témája, keymapje és ablakmérete módosíthatja az elhelyezést. A felépítési ábra szemléltetés, nem élő képernyőkép.

Dokumentum frissítése: 2026. szeptember 17. Ez a korábbi rövid magyar PDF kibővített változata; tartalmazza a Claude/MCP útmutató lényeges lépéseit is. Az angol kiadás ugyanezt a funkciókört mutatja be.

Elérési út az IDE-ben: Code > Spring Boot Debug REPL and MCP > Help (PDF) > Magyar vagy English. A Java-forrás helyi menüje ugyanezt kínálja. A Java REPL > Tools > Help (PDF) nyelvválasztót nyit. Mindkét kézikönyv a plugin része, offline is elérhető.

12	82	200 MiB
fő fül, gombmagyarázatokkal	MCP-eszköz Claude számára	alap DATA snapshotkorlát



Tartalomjegyzék

A fejezetcímekre kattintva közvetlenül a megfelelő oldalra léphetsz. A PDF-olvasó könyvjelzőpanelje ugyanezt a szerkezetet mutatja.

01. Első indítás és profilok . . . 4
02. A felület térképe . . . 5
03. Java REPL: kapcsolat és futtatás . . . 6
04. Java REPL: a munkafüzet gombjai . . . 7
05. Eredmény, JSON és kódellenőrzés . . . 8
06. Variables: a session változói . . . 9
07. Snapshotok: mit ments el? . . . 10
08. Snapshots > Saved: gombok és betöltés . . . 11
09. Snapshots > Capture next . . . 12
10. Snapshots > Compare . . . 14
11. Inspector: az objektumböngésző . . . 15
12. Tap / Trace: megfigyelés futás közben . . . 16
13. Cases / Reload: teszteset készítése . . . 17
14. Cases / Reload: futtatás és HotSwap . . . 18
15. Debugger: lokális értékből REPL-objektum . . . 19
16. HTTP: mentett kérés összeállítása . . . 20
17. HTTP: gombok és végrehajtás . . . 21
18. AI: promptból Java-javaslat . . . 22
19. Imports és pluginbeállítások . . . 23
20. MCP: a helyi kiszolgáló gombjai . . . 24
21. Claude Code: kapcsolódás és első feladat . . . 26
22. Claude Desktop és másik gép . . . 27
23. Teljes munkamenet: rögzítésből javítás . . . 28
24. Gyorsbillentyűk és menük . . . 29
25. MCP-eszközreferencia Claude-nak . . . 30
26. Hibaelhárítás a felületről . . . 34
27. Korlátok, fogalmak és további anyagok . . . 35
28. Futtatási mód és tranzakció . . . 36
29. Reprodukció egy csomagban . . . 37
30. Hibát elváró CASE és capture-metaadat . . . 38
31. Audit, titkok és további fejlesztések . . . 39
32. CASE 2.0: mezők és assertionök . . . 40
33. Paraméterezés, életciklus és eredmény . . . 42
34. CASE-ből JUnit 5 teszt . . . 43
35. Notebook: cellaállapot és függőségek . . . 44
36. Teljes workspace mentése és megnyitása . . . 46
37. Snapshot-verziók és származási adatok . . . 47
38. Snapshotpont egérrel a Java-forrásban . . . 48
39. Rögzített hívásfa és értékek a forrás mellett . . . 49
40. AI-agentek hozzáférése a hívásgráfhoz . . . 54
41. SQL, N+1-jelzés és DB-ig követhető hívások . . . 56
42. Hibernate a rögzített hívásfolyamban . . . 59
43. Rögzített hívásból futtatható CASE . . . 62

44. DATA-másolat szerkesztése és variációk 63
45. Bean Explorer 64
46. Watch és mezőváltozások 65
47. Aszinkron flow és tranzakcióhatárok 66
48. HotSwap és érintett CASE-ek áttekintése 67
49. MCP: új eszközök, események és feladatok 68

01. Első indítás és profilok

Cél: a saját Spring Boot alkalmazásod sikeresen felálljon, és a REPL-ben használható legyen a `ctx`.

1. Telepítsd a `build/distributions/sb-repl-0.24.1.zip` csomagot: Settings > Plugins > Install Plugin from Disk. Pluginfrissítés után indítsd újra az IDEA-t.
2. Nyisd meg a szokásos Spring Boot Run Configurationt. Pipáld be az **Enable Spring Boot Debug REPL and MCP** opciót; a normál konfiguráció main classát, környezetét és VM-paramétereit használd.
3. Spring Boot konfigurációnál az **Active profiles** mezőbe például `dev,llm-openai` kerül. Sima Application konfigurációnál ezt a programargumentumot használd:

```
--spring.profiles.active=dev,llm-openai
```

4. Indítsd az alkalmazást **Run** vagy **Debug** módban. A debuggerfunkciókhoz Debug kell; a sima REPL-hez Run is elég.
5. Nyisd meg a **Spring Boot Debug REPL and MCP** tool window-t, és várd meg a **READY** állapotot. A Java REPL fülön futtasd:

```
ctx.getBeanDefinitionCount()
```

Várt eredmény: egy szám a Value panelen. Ez igazolja, hogy a kifejezés elérte a bekötött Spring contextet. A teljes alkalmazás üzleti működését ez önmagában nem teszteli.

Melyik indítási mód mire való?

Mód	Mikor használd?
Normál Spring Boot + REPL-checkbox	Ajánlott napi használat; megtartja a Spring profil- és környezeti beállításait.
Application + REPL-checkbox	Java-main indítása; a Spring-profilokat argumentummal vagy környezettel adod meg.
Tools > Attach & Inject Dev Runtime	Már futó, általad kiválasztott JVM-hez csatlakozás. Késői attach esetén a bridge segíthet a context megtalálásában.
Régi, külön Spring Boot Debug REPL and MCP konfiguráció	Kompatibilitás miatt megmaradt; új beállításnál a normál konfigurációt válaszd.

A checkboxos indítás a plugin beépített agentjét használja; nem kell külön alkalmazásfüggőség. Alkalmazáskódba írt `SnapshotHelper` hívásokhoz viszont kell a bridge. A puszta `dev,llm-openai` argumentum nem aktivál profilokat. Az nREPL elindulása után bekövetkező Spring-konfigurációs hibát az alkalmazásban kell kijavítani.

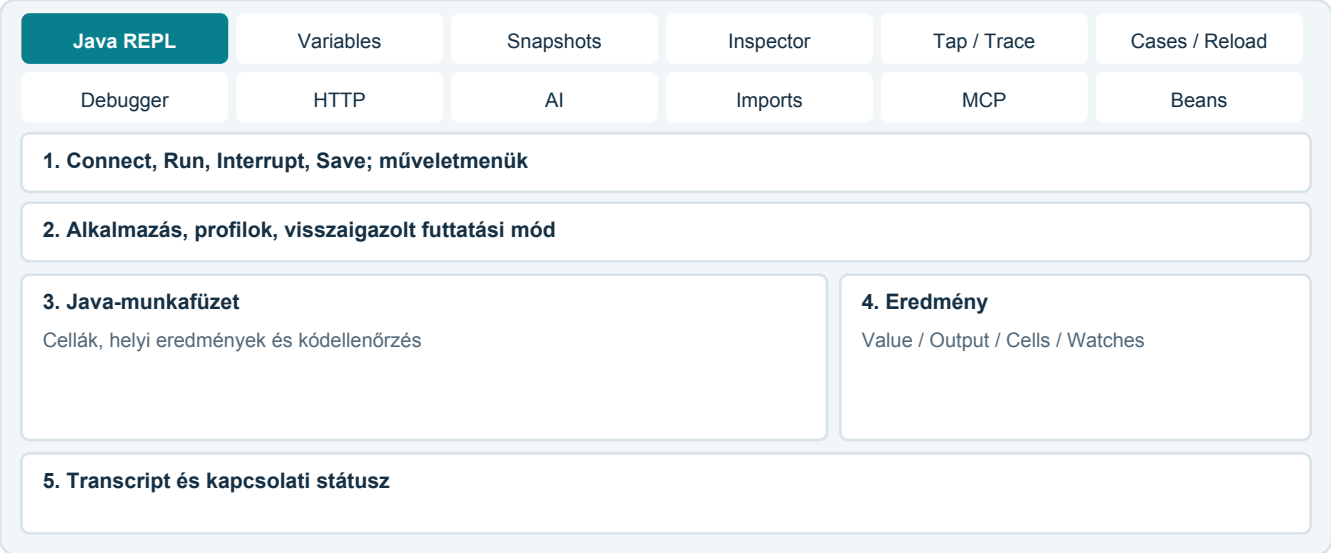
A kapcsolat állapotai

DISCONNECTED: nincs kapcsolat. **CONNECTING:** kapcsolatfelépítés. **SESSION_READY:** használható Java-session, de a Spring még nem feltétlenül kész. **WAITING_CONTEXT:** contextre várakozó állapot. **READY:** Spring bekötve. **FAILED:** kapcsolatfelépítési hiba; olvasd el a részletes státuszt és az alkalmazás logját. Futás közben **Queued / running** is megjelenhet.

02. A felület térképe

A fő fülek balról jobbra ebben a sorrendben találhatók. A Java REPL fülön négy fő ikon és négy műveletmenü hagy több helyet a kódnak. Szűk ablakban a fülsor több sorba kerülhet; a kód és az eredmény közti osztó húzható.

Fül	Mire való?
Java REPL	Kapcsolat, Java-munkafüzet, futtatás, ellenőrzés és eredmény.
Variables	A saját IDE-session változóinak listája, beillesztése és inspectálása.
Snapshots	Saved, Capture next és Compare alfülek; tartós adatok és capture-szabályok.
Inspector	Valódi objektumok navigálható böngészője; Value és Fields alfül.
Tap / Trace	Alkalmazásértékek megfigyelése és metódushívások követése.
Cases / Reload	Snapshotból mentett próba, elvárt eredmény, ismételt futtatás és HotSwap.
Debugger	Az IDEA aktuális debug-sessionjének kezelése, lokális érték átvétele.
HTTP	Mentett HTTP-kérések összeállítása, futtatása és snippetgenerálás.
AI	Áttekinthető prompt küldése a beállított API-nak; kódjavaslat készítése.
Imports	Mentett importbeállítások; utólag alkalmazhatók a sessionre.
MCP	Helyi MCP-kiszolgáló külső AI-klienseknek, például Claude-nak.
Beans	Beandefiníciók, függőségek, proxy-metaadatok és hívás-előkészítés.



Eredmény: Value > Tree / Formatted / Raw: érték; Output / errors: kimenet és hibák; Cells: cellafutások; Watches: rögzített megfigyelések. **Inspector:** Value: strukturált érték; Fields: élő objektumnavigáció.

03. Java REPL: kapcsolat és futtatás

Hely: Java REPL, felső eszköztár. A négy fő ikon: **Connect**, **Run cell / selection**, **Interrupt**, **Save workspace**. Az ikon fölé vitt egér megmutatja a nevét. A mellettük levő **Run**, **Workspace**, **Session**, **Tools** menü tartalmazza a részletes műveleteket. A Connect egy ismert vagy a Settingsben kiválasztott endpointhoz kapcsolódik; nem indít Spring-alkalmazást.

Hol találom? Run: cellák futtatása és függőségvizsgálat. Workspace: munkafüzet, mentés, history. Session: kapcsolat, Bind, Reset és Execution settings. Tools: ellenőrzés, beanek, importok, reprodukció, audit, PDF és MCP. A részletes hiba és transcript alul marad; az alkalmazás/profil és a tényleges futtatási mód felül látszik.

Gomb / kapcsoló	Mit csinál, és mi kell hozzá?
Connect	Kapcsolódik az ismert agent-endpointhoz. Élő kapcsolat esetén nem nyit új sessiont.
Open endpoint	Egy helyi `.properties` endpointfájlt választasz, majd kapcsolódik a benne megadott agenthez. Nem application.yml-t kell kiválasztani.
Disconnect	Lezárja a REPL-kapcsolatot és a hozzá tartozó sessiont. A Spring-folyamatot nem állítja le. Az MCP-elérés is megszűnik.
Bind	Megpróbálja bekötni az időközben felállt Spring contextet. Siker esetén READY; contextcsere után Reset szükséges.
Run cell / selection	A kijelölt szöveget futtatja. Kijelölés nélkül a kurzor aktuális celláját.
Run + next	Ugyanez, majd siker után a következő meglévő cellára lép, ha közben nem változott a szöveg és a kurzor.
Run all	A nem üres cellákat sorban futtatja, saját eredménnyel. Hiba, szerkesztés vagy megszakítás után a sorozat megáll. A korábbi cellák hatásai ismét megtörténhetnek.
Interrupt	A saját futó kiértékelés megszakítását kéri. Nem visszagörgetés; a kód együttműködése szükséges.
Reset	Megerősítés után eldobja a session deklarációit, handle-jeit és LIVE pinjeit, majd az aktuális contexthez új sessionállapotot épít. DATA megmarad.
Complete	Kiegészítést kér az aktuális kurzornál. Nem értékeli ki a kódot.
Tools > Toggle live check	Be- vagy kikapcsolja az automatikus, futtatás nélküli kódellenőrzést. Alapból be van kapcsolva.
Check code	Azonnal kéri a munkafüzet elemzését.
Help (PDF)	Magyar vagy angol offline kézikönyvet választhatsz. Nincs szükség REPL-kapcsolatra.
MCP	Átvált az MCP fülre. Önmagában nem indít MCP-szervert.

04. Java REPL: a munkafüzet gombjai

Hely: Java REPL > Workspace menü; az Insert bean és Apply configured imports a Tools menüben van. A forrás és az élő session állapota eltérhet; a 35. fejezet a cellaállapotot és a Run menüt, a 36. a workspace-műveleteket részletezi.

Gomb	Hatás
Insert cell	Az aktuális cella végére új `// %%` határt szúr be, és oda állítja a kurzort.
Open workbook	Helyi UTF-8 fájllal lecseréli a munkafüzet szövegét. Legfeljebb 1 000 000 bájt. Nem futtat kódot és nem reseteli a sessiont.
Save workbook	`.jsh` fájlba menti az aktuális szöveget. Nem menti a Java-objektumokat.
Save RECIPE	A kijelölést, ennek hiányában a teljes munkafüzetet névvel a runtime tárába menti. Nem futtat.
History	Az utolsó legfeljebb 100 snippetből választhatsz; a kiválasztott kódot a kurzorhoz illeszti.
Clear history	Törli az előzménylistát és a Java REPL alsó transcriptjét. Nem dobja el a változókat.
Insert bean	Bean-nevet/típust választasz, opcionális publikus típust adsz meg, majd egy `ctx.getBean(...)` deklarációt illeszt be. Csak futtatáskor kéri le a beant.
Apply configured imports	Az Imports fülön engedélyezett teljes osztályneveket importálja az aktuális sessionbe.

Két cellás, kipróbálható munkafüzet

```
// %% Előkészítés
import java.util.List;
var numbers = List.of(2, 4, 6);

// %% Kísérlet
numbers.stream().mapToInt(Integer::intValue).sum()
```

Előbb futtasd az első, majd a második cellát. A második eredménye `12`. Ha később csak a második cellát módosítod, nem kell újra előállítanod a bemenetet.

Cellahatár: a `// %%` külön sorban álljon; utána cím következhet. Stringen, text blockon vagy blokkkommenten belüli ilyen szöveg nem osztja fel a munkafüzetet. Határ nélküli fájl egyetlen cella.

Állapotmegőrzés: a Save workbook / Save RECIPE a kódot őrzi. A DATA az értéket őrzi. Újracsatlakozás után egy munkafüzet megnyitása nem hozza automatikusan létre a változókat; a szükséges előkészítést futtasd le.

05. Eredmény, JSON és kódellenőrzés

Hely: Java REPL, jobb oldali eredménypanel és a szerkesztő alatti ellenőrzési státusz.

Elem	Mit mutat vagy hajt végre?
Value > Tree	Összecsukható objektummezők, mapek, listák, tömbök, típusok, nullok és ismételt referenciák.
Value > Formatted	Behúzott olvasható szöveg; JSON esetén kétszóközös formázás.
Value > Raw	Az elérhető nyers eredményszöveg; JSON-stringnél az eredeti szöveg. Ez is lehet korlátozott előnézet.
Expand / Collapse	A megjelenített fa kibontása vagy összecsukása. Nem kér le tetszőleges mélységben új objektumadatot.
Copy formatted	A formázott megjelenítést másolja a vágólapra.
Output / errors	A kiértékelés stdout/stderr kimenete és hibái.
Inspect result	A legutóbbi használható eredményhandle objektumát megnyitja az Inspectorban. A kifejezést nem futtatja újra.
Pin LIVE	Az aktuális eredmény élő referenciáját menti a megadott néven.
Freeze DATA	Az aktuális eredményből tartós adatot készít; nevet és opcionális deklarált típust kér.

Automatikus JSON-felismerés: objektumot vagy tömböt tartalmazó érvényes JSON-stringből fa és behúzott megjelenítés készül. Hibás JSON szövegként jelenik meg. A **Partial preview** nem azt jelenti, hogy az eredeti objektum csonka: a megjelenítés érte el a korlátot.

Ellenőrzés és kiegészítés

Live check: 650 ms gépelési szünet után elemzi a munkafüzetet. Hiba és figyelmeztetés aláhúzással jelenik meg; az üzenethez vidd fölé az egeret. A korábbi cellák deklarációit elemzési kontextusként ismeri, akkor is, ha azok még nem futottak le. Nem hívja meg az inicializálókat és nem módosítja az élő sessiont.

Complete / Ctrl+Space: az aktuális JShell-session alapján ad javaslatokat; azonosító vagy pont gépelése után 300 ms késleltetéssel is kérhet kiegészítést. A deklarációs cellát előbb futtasd le. Emiatt lehetséges, hogy az ellenőrző már ismer egy változót, de a futó session kiegészítője még nem.

Ez fordítói szintaktikai/típusellenőrzés, nem teljes üzleti validáció vagy minden szabályt lefedő linter. A `last1`, `last2`, `last3` a közelmúlt eredményobjektumai, a `lastError` a legutóbbi kiértékelési hiba.

06. Variables: a session változói

Hely: a második fő fül. A lista a runtime által visszaigazolt IDE-sessionváltozókat mutatja. A fül megnyitásakor automatikusan frissül.

Gomb	Használat és hatás
Refresh	Újra lekéri a változólistát. Lezárt kapcsolat esetén üríti a nézetet.
Insert	A kijelölt változó nevét beilleszti a Java REPL kurzorához, és a munkafüzetre vált. Nem futtat.
Inspect	A kijelölt változó objektumát megnyitja az Inspectorban.
Previous / Next	A jelenlegi főablak-bekötésben ezek is az Inspectort nyitják meg a kiválasztott változóval; a tényleges lapozáshoz az Inspector saját gombjait használd.
Drop	Eldobja a kijelölt Java-változó deklarációját a sessionből, majd frissíti a listát. Nem törli az adatbázisbeli adatot vagy a mentett DATA snapshotot.

Mikor melyik művelet hasznos?

1. Kiértékeltél egy nagy listát `items` néven, de nem szeretnéd újra lekérni: **Variables > items > Inspect**.
2. Egy következő cellában ugyanarra az értékre hivatkoznál: **Insert**, majd egészítsd ki például `items.size()` kifejezéssé és futtasd.
3. Egy kísérleti változót már nem használsz: **Drop**. Más objektum vagy bean ettől még tarthat referenciát ugyanarra az értékre; a memóriafelszabadulás nem azonnali ígéret.

Mire nem szolgál ez a lista?

Nem az alkalmazás összes lokális változóját mutatja, és nem a Spring beanlista. Metóduson belüli értékhez a **Debugger > Capture to REPL** vagy egy alkalmazáskódbeli capture/tap pont kell. A beaneket **Java REPL > Insert bean** segítségével választhatod ki.

Claude saját MCP-sessionjének változói nem jelennek meg itt. Az IDE és Claude közötti átadáshoz ments DATA snapshotot, majd a másik sessionben töltsd be.

07. Snapshotok: mit ments el?

Hely: Snapshots. Az adat típusától és kívánt élettartamától függ, melyik mentési módot választod.

Mód	Mit őriz meg?
LIVE	Az eredeti objektumreferenciát az adott sessionben, legfeljebb 30 percig. Későbbi módosításai látszanak. Session- vagy contextlezáráskor elvész.
DATA	Verziózott JSON-burkolatban a kiválasztott adatot. Fájlban megmarad; betöltéskor új objektum készülhet belőle.
RECIPE	Java-forrást, például előkészítő cellákat. Betöltése beilleszt, a Run hajtja végre.
CASE	Bemeneti és elvárt DATA-nevet, Java-kódot és bemeneti típusbeállítást. A Cases / Reload fülön futtatható.

Adat, típus és méret

DATA-hoz Jackson kell az alkalmazásban. A mentés a meglévő ObjectMapper külön másolatával vagy külön mapperrel dolgozik; az alkalmazás mapperét nem módosítja. A mentés gettereket, a visszatöltés konstruktorokat és deszerializációs kódot hívhat.

DTO-t, rekordot vagy célzott projekciót ments. Teljes Spring bean, kapcsolat, Hibernate proxy vagy a teljes alkalmazásobjektum-gráf helyett válaszd ki a reprodukcióhoz szükséges adatot. A kizárólag JShellben deklarált osztály egy másik sessionben vagy újraindítás után nem feltétlenül áll rendelkezésre.

A fájlkorlát alapból **200 MiB**, a metaadatokat is beleértve. Nagy adat betöltése ennél több heapet igényelhet. Fájlhoz **Import file / Export file**, kis beillesztett JSON-hoz **Paste JSON** való. Az IDE inline JSON-import korlátja 2 MiB; az MCP-é ennél kisebb.

Gyűjtemények és mixinek

A **Freeze DATA** típusmezőjében add meg a generikus elemtípust, például `java.util.List<com.example.ItemDto>`. Így visszatöltés után az elemek típusa is megőrizhető.

```
import com.baader.devrt.SnapshotManager;
SnapshotManager.save("item-list", items,
    "java.util.List<com.example.ItemDto>");
```

A `com.example` neveket a saját alkalmazásod osztályaira cseréld. Snapshothoz külön Jackson mixin adható: `SnapshotManager.addMixIn(Target.class, Mixin.class)`. Ez a session snapshot-szerializációját módosítja. Amit a mixin elhagy, azt a későbbi betöltés sem fogja helyreállítani.

A DATA nem teljes alkalmazásmentés. Nem ment hívási vermet, tranzakciót, adatbázist vagy nyitott kapcsolatokat; visszatöltése nem állítja vissza a Spring beanek állapotát.

08. Snapshots > Saved: gombok és betöltés

Hely: Snapshots > Saved. A sor a nevet, a `[LIVE]`, `[DATA]`, `[RECIPE]` vagy `[CASE]` módot és a típust jelzi. Több sor kijelöléséhez Ctrl/Cmd-kattintást használhatsz.

Gomb	Mire való, és milyen adatot kér?
Refresh	Frissíti az elérhető mentések listáját.
Pin LIVE	Snapshotnevet és meglévő Java-változónevet kér. Üres változómező az utolsó eredményt használja.
Freeze DATA	Ugyanez tartós adatmentéssel, plusz opcionális teljes Java-típussal. Kifejezés helyett már létrehozott változót adj meg.
Load	A kijelölt DATA/LIVE értéket a megadott REPL-változóba tölti; alapnév `restored`. RECIPE esetén csak a forrást illeszti be.
Info	Megmutatja a kijelölt mentés típusát, módját, idejét és méretinformációit.
Paste JSON	Snapshotnevet és JSON-szöveget kér, majd DATA-ként importálja. Nem értékel ki Java-kódot.
Import file	Helyi JSON-fájlt és snapshotnevet kér; a runtime végzi a fájlimportot.
Export file	A kijelölt tartós mentést fájlba exportálja. Meglévő fájlnál felülírás-megerősítést kér. LIVE-ot előbb DATA-ként ments.
Compare	Pontosan két DATA kijelölése után a Compare alfülre vált és összehasonlítja őket.
Delete	Megerősítés után törli a kijelölt mentést. Közös DATA-t más session is használhat.

Példa: egy újra használható bemenet

1. A Java REPL-ben hozz létre egy `input` változót, vagy vedd át debuggerből.
2. **Saved > Freeze DATA**: név `cv-input-42`, változó `input`, típus a saját DTO-d teljes neve. Az üres opcionális típusmezőt OK-val fogadd el, ne Cancel-lel.
3. Később jelöld ki a mentést, **Load**, változónév `restoredInput`.
4. **Variables > Refresh > restoredInput > Inspect** segítségével nézd meg a visszatöltött értéket.

A Load gomb nem kér külön típusfelülírást: a DATA-ban elmentett típust használja. Eltérő típusra a runtime/MCP megfelelő betöltési API-ja használható. A listában szereplő CASE-t a **Cases / Reload > Load selected** gombbal nyisd meg.

Helyi fájlok: az Import file / Export file fájlútvonalát a futó alkalmazás használja. Az IDE és az alkalmazás ugyanazt a fájlrendszert kell lássa. Azonos névvel az aktuális érték új verzióra vált; a korábbi érték a Versions alatt megmarad. A törlés a verziókat is eltávolítja.

09. Snapshots > Capture next

Cél: egy következő valódi alkalmazáshívásnál automatikusan mentsen egy kiválasztott értéket. Az élesítés önmagában nem indít HTTP-kérést vagy üzleti folyamatot.

Mező / gomb	Jelentés
Capture point	Az alkalmazáskódba írt pont neve, például `cv-input`. Pontos egyezés szükséges.
Snapshot name (creates a new version)	Az elkészülő DATA neve. Azonos névnél új tartós verziót hoz létre.
Exact case ID filter (empty = any)	A capture hívás `caseId` paraméterének pontos szűrője; üresen bármelyiket fogadja. Nem mentett CASE neve.
Declared type (optional)	A mentendő adat teljes, opcionálisan generikus Java-típusa.
Arm capture rule · 5 minutes	Öt percre új, független szabályt élesít. A megadott számú mentés után kikapcsol.
Disarm selected rule	A táblában kijelölt saját várakozó szabályt visszavonja. Már folyó szerializációt nem szakít meg.
Capture count	1-100 mentés. Több mentésnél automatikus sorszám, vagy a névben `${sequence}` helyőrző.
Capture every Nth matching call	1-10000; az első találatot, majd minden N-edik további találatot menti.
Refresh status	Azonnal lekéri az állapotot és a saját szabályok tábláját. Kattints egy szabály sorára a kijelöléshez.

Az alkalmazáskód előkészítése

A bridge-nek a forrás fordításakor is elérhetőnek kell lennie. A plugin repositoryjában telepítsd helyi Maven-tárba:

```
mvn -f sb-repl-bridge/pom.xml install -Dgpg.skip=true
```

Az alkalmazás fejlesztési függősége: `hu.baader:sb-repl-bridge:0.24.1`. A checkboxos agent önmagában nem biztosítja a Java-forrás fordításához szükséges importot. A feldolgozó metódusban:

```
import com.baader.sbrepl.bridge.SnapshotHelper;
SnapshotHelper.capture("cv-input", requestId, inputDto);
```

A `requestId` String; az `inputDto` a mentendő adat. Drága projekciónál `captureLazy("cv-input", requestId, () -> projectToDto(inputDto))` használható. A supplier csak élesített, egyező találatnál fut le egyszer. A mentés szinkron, a hívó szálon történik.

Az állapot olvasása

ARMED: találatra vár. **CAPTURING:** a mentést egy hívás már lefoglalta. **SAVED:** siker, mérettel és időtartammal. **FAILED:** hiba, részletekkel. **EXPIRED:** lejárt. **CANCELLED:** visszavonva. **OTHER_SESSION:** nincs saját szabály, de más sessionben van aktív. JVM-enként legfeljebb 16 független szabály lehet aktív. Mentés közben az adott szabály további találatai kimaradnak; az átfedő szabályok közös supplierje egy alkalmazáshívásban egyszer fut. A hibás szabály leáll.

Siker után a mentés a Saved listán elérhető. Élesítés nélkül vagy agent hiányában a capture `false` értéket ad; a sima `SnapshotHelper.save(...)` viszont feltétel nélkül ment és hibát dobhat.

10. Snapshots > Compare

Belépés: a Saved alfülön jelölj ki pontosan két DATA snapshotot, majd **Compare**. A Before és After sorrendjét a megjelenő címben ellenőrizd.

Elem	Működés
Before / After cím	A két összehasonlított mentés neve.
Field / array index	A különböző mező vagy tömbelem elérési útja.
Change	`ADDED`, `REMOVED` vagy `CHANGED`.
Before / After oszlop	Az eltérő értékek korlátozott előnézete.
Previous / Next	Száz eltéréses oldalakon lép. A következő gomb csak további eredménynél aktív.
Swap before / after	Megfordítja az összehasonlítás irányát és az első oldalról újraszámolja.

Hogyan értelmezd az eredményt?

No DATA differences: nem talált tartalmi eltérést az elvégzett vizsgálatban. A mentési idő és a burkolat metaadatai nem számítanak eltérésnek. Hiányzó mező és `null` külön értéknek számít.

Partial comparison: elérte a bejárási korlátot vagy megszakadt. Az eddig talált eltérések használhatók, de a nulla találat ilyenkor nem bizonyít egyenlőséget. Az értékek előnézete rövidített lehet.

Javasolt munkamenet

1. A javítás előtt mentsd az eredményt `cv-before` néven.
2. A javítás után ugyanarra a bemenetre ments `cv-after` DATA-t.
3. Jelöld ki a két mentést, és nyomj Compare-t.
4. A szándékolt mezőváltozások mellett ellenőrizd a váratlanul hozzáadott vagy eltűnt mezőket is.

Az összehasonlítás mentett JSON-adaton dolgozik, nem tölti vissza a DTO-kat és nem futtatja újra a service-hívást. A panelen látott cellaszöveg előnézet; a teljes objektum mélyebb elemzéséhez töltsd be a szükséges snapshotot és használd az Inspectort.

11. Inspector: az objektumböngésző

Megnyitás: Java REPL > Inspect result, Variables > Inspect, Tap / Trace > Inspect selected, debugger capture vagy CASE > Inspect actual. Az Inspector a kiválasztott élő objektumhoz kapcsolódik.

Gomb / mező	Működés
Value	Az aktuálisan megnyitott objektum Tree / Formatted / Raw előnézete.
Fields	Mezők/elemek táblázata: Field / index, Type, Preview és Access.
Open selected / dupla kattintás	Belép a Fields táblában kijelölt objektummezőbe vagy elembe. Access-hiba esetén annak magyarázatát mutatja.
Back	Visszalép a korábban megnyitott objektumhoz.
Refresh	Újraolvassa az aktuális objektum mezőoldalát.
Previous / Next	Ötven elemű oldalakon lép. A nagyobb konténerekre bejárási korlát vonatkozik.
Variable	A Bind current célváltozója; alapból `inspected`.
Optional Java type	Opcionális publikus/deklarált típus kötéshez és DATA-mentéshez.
Bind current	Az aktuálisan megnyitott valódi objektumot a megadott REPL-változóhoz köti.
Snapshot	A mentés neve; alapból `inspected-data`.
Pin LIVE / Freeze DATA	Az aktuálisan megnyitott objektumból élő pin vagy tartós adatmentés készül.

Fontos különbség: a Value-fa kibontása csak a megjelenítést változtatja. A **Bind current** és a mentés célját a **Fields > Open selected** navigáció változtatja. Előbb lépj be a kívánt objektumba, majd kösd változóhoz.

Példa: egy elem kiemelése egy válaszból

1. Nyisd meg a service-hívás eredményét az Inspectorban.
2. A Fields lapon keresd meg az `items` mezőt, majd Open selected.
3. Lapozz, válassz egy elemet, és ismét Open selected.
4. Variable: `selectedItem`; Bind current.
5. A Java REPL-ben az új változóval dolgozhatsz. Tartós mintához Freeze DATA.

Az automatikus előnézet közvetlen mezőket olvas; nem futtatja újra az eredeti üzleti kifejezést. Explicit navigációnál például egy saját kollekció hozzáférő kódja futhat. Reset vagy contextcsere után új értéket kell megnyitni; a régi handle érvénytelen.

12. Tap / Trace: megfigyelés futás közben

Tap: az alkalmazás kijelölt helyei értékeket adnak át. **Trace:** egy megnevezett metódus hívásait követed. Mindkettő élő referenciákat tart meg rövid ideig.

Az alábbi vezérlők a **Live events** alfülön vannak. A **Recorded calls** alfül osztályok tényleges metódushívásaiból hívásfát és menthető, forrás mellett megnyitható értékfelvételt készít; a használatát a 39. fejezet mutatja be.

Mező / gomb	Hatás
Exact tap label (empty = all)	Üresen minden tap; megadva csak a pontos címke. A Start tap alkalmazza.
Start tap / Stop tap	Elindítja vagy leállítja a saját session tap-fogadását.
Refresh	Frissíti az eseményeket és a bekapcsolt trace-ek listáját.
Clear values	Törli a megtartott eseményértékeket. A tap-fogadást és a trace-konfigurációt külön kell leállítani.
Inspect selected / dupla kattintás	Az eseményhez tartozó értéket megnyitja az Inspectorban.
Class	A követendő alkalmazásosztály teljes neve.
Method (all overloads)	A metódus neve; az azonos nevű deklarált túlterhelésekre is vonatkozik.
Trace / Untrace	Bekapcsolja vagy kikapcsolja a megadott metódus követését.
Stop all traces	Leállítja a saját session összes követését.

A táblázat oszlopai: **ID**, **Time**, **Kind**, **Label / method**, **ms**, **Preview**. Az alsó trace-listán kiválasztott sor visszatölti az osztály- és metódusnevet a mezőkbe. Látható panelnél az eseménylista automatikusan is frissül.

Tap beillesztése a saját kódba

```
SnapshotHelper.tap("cv-input", inputDto);
SnapshotHelper.tap("cv-result", resultDto);
```

Ehhez a bridge-függőség kell. Az agent hiánya vagy a feliratkozók hiánya esetén a tap nem ment semmit. A tap önmagában nem szerializál és nem készít DATA-fájlt.

Trace használata kódmódosítás nélkül

Állj a normál Java-forrásban egy metódusra: **Code > Spring Boot Debug REPL and MCP > Trace Method**. Ez a fül Class/Method mezőivel is elvégezhető. A követésben argumentum, visszatérési érték, kivétel, szál és időtartam válhat vizsgálhatóvá.

Sessionönként legfeljebb **128 eseményérték**, **öt percig** marad meg. A `dropped` számláló kihagyott eseményeket jelezhet; ez nem teljes auditnapló. A fontos értéket az Inspectorból mentsd DATA-ként, majd a szükségtelen megfigyelést állítsd le.

13. Cases / Reload: teszteset készítése

A 0.15-ös assertion-, paraméterezési és JUnit-export mezőket a 32-34. fejezet mutatja be. Az alábbi egyszerű CASE-formátum továbbra is használható.

Cél: ugyanazt a rögzített bemenetet újra lefuttatni, és az eredményt egy elvárt DATA snapshothoz hasonlítani. A CASE itt fejlesztői próba a futó alkalmazásban; nem helyettesíti a projekt automatizált tesztjeit.

Mező / gomb	Jelentés
Case name	A mentett eset neve.
Input DATA	A bemenet snapshotja; a lista DATA-elemeket kínál.
Expected DATA	Az összehasonlításhoz használt elvárt snapshot.
Variable	A bemenet változóneve a próba saját evaluatorában; alapból `input`.
Optional input type	A betöltendő bemenet opcionális, teljes Java-típusa.
Java-kód mező	A bemeneten futtatandó kód. Az utolsó előállított értéket hasonlítja az elvárthoz.
Refresh	Frissíti a CASE-listát és a választható DATA-neveket.
Save case	Elmenti a mezők és a kód aktuális értékét. Nem futtatja a próbát.
Load selected	A kijelölt CASE definícióját szerkesztésre betölti. Nem futtat.

Egy rövid, önálló példa

A Java REPL-ben készíts bemenetet és elvárt értéket, majd mindkettőt Freeze DATA-val mentsd:

```
var numbers = new java.util.ArrayList<Integer>(
    java.util.List.of(1, 2, 3));
int expectedSum = 6;
```

Bemenet: `demo-input`, típus: `java.util.List<java.lang.Integer>`. Elvárt érték: `demo-expected`, változó: `expectedSum`. Ezután a Cases / Reload mezői:

• Case name: `demo-sum`; Input DATA: `demo-input`; Expected DATA: `demo-expected`.

• Variable: `input`; Optional input type: `java.util.List<java.lang.Integer>`.

• A kód:

```
input.stream().mapToInt(Integer::intValue).sum()
```

Nyomj **Save case**, jelöld ki az esetet, majd **Run saved cases**. A várt kimenet `PASSED`.

A CASE-kód a saját, friss evaluatorában fut. A munkafüzetben létrehozott `processor` változó nem kerül át automatikusan: a CASE-ben szükség esetén `ctx.getBean(...)` segítségével kérd le a service-t.

14. Cases / Reload: futtatás és HotSwap

Hely: ugyanazon a fülön a vezérlőgombok és a CASE-eredménytábla. Egy futásban 1-20 elmentett eset jelölhető ki.

Gomb	Hatás
Run saved cases	Sorban futtatja a kijelölt, elmentett definíciókat. A még el nem mentett űrlapmódosítást előbb Save case-szel rögzítsd.
Rerun failed	Az eddig kapott FAILED, ERROR és INCONCLUSIVE eseteket futtatja újra.
Stop	Megállítja a további workflow-lépéseket; futó kiértékelésre megszakítást kérhet. Külső hatást nem von vissza.
Inspect actual	A kijelölt eset legutóbbi tényleges eredményét az Inspectorban nyitja meg, ha van hozzá eseményérték.
Use open Java editor	A normál szerkesztőben megnyitott `.java` forrást választja ki reloadhoz; az aktuális, akár még nem mentett tartalommal dolgozik.
Choose Java file	Másik `.java` fájlt választasz; minden futás a legfrissebb tartalmat olvassa.
Reload + run selected	Fordít és HotSwapot kér, majd csak sikeres frissítés után futtatja a kijelölt eseteket.

Eredmény	Értelmezés
NOT RUN	Még nincs futtatási eredmény.
PASSED	A tényleges érték egyezik az elvárt DATA-val.
FAILED	Tartalmi eltérés van. Nézd meg a részletes kimenetet.
ERROR / CANCELLED	Hiba, illetve megszakított végrehajtás.
INCONCLUSIVE	A vizsgálat nem tudott dönteni, például összehasonlítási korlát miatt. Nem tekinthető sikeres tesztnek.

Mit jelent az állapotmegőrző kódfrissítés?

A szabványos JVM által támogatott metódustörzs-változás után az alkalmazás meglévő objektumai megmaradnak. Új mező, új metódus vagy szerkezeti változás újraindítást igényelhet. A HotSwap **kézzel indított** művelet; a forrás mentése önmagában nem frissíti a JVM-et.

A normál Java-forrás **Reload Class** menüpontja csak a frissítést végzi. A **Reload + run selected** a frissítéshez a kiválasztott CASE-eket is hozzákapcsolja. Hiba után ne feltételezd, hogy már az új kód fut.

A friss evaluator ellenére a CASE valódi Spring beaneket ér el. Az adatbázisírás, HTTP-hívás és beanállapot-változás megmaradhat a futás után.

15. Debugger: lokális értékből REPL-objektum

Előfeltétel: az alkalmazás Debug módban fut, a REPL-checkbox aktív. A fül az IDEA aktuális debug-sessionjét és kiválasztott stack frame-jét használja.

Gomb / mező	Működés
Pause	Megállítást kér a futó debug-sessiontől.
Resume	Folytatja a felfüggesztett programot.
Step over / Step into / Step out	Az IDEA szokásos léptetései a megállított sessionben.
Show stack / locals	Megnyitja az IDEA Debug ablakát és megmutatja a végrehajtási pontot. A frame-et ott válaszd ki.
Java expression mező	A kijelölt frame-ben értelmezendő kifejezés, például `input` vagy `this`.
Evaluate in frame	A megállított frame környezetében kiértékeli és a fülön megjeleníti a kifejezést.
REPL variable	Az átvett érték változóneve; alapból `debugValue`.
Capture to REPL	Egyszer kiértékeli és átadásra megőrzi az objektumot; Resume után veszi át a REPL-session.

Lépések egy service lokális bemenetéhez

1. Tegyél breakpointot oda, ahol az `inputDto` már rendelkezésre áll.
2. Indítsd a szükséges alkalmazáskérést; a debugger megáll.
3. **Show stack / locals** segítségével válaszd ki a megfelelő frame-et.
4. A kifejezésmezőbe írd: `inputDto`. REPL variable: `capturedInput`.
5. **Capture to REPL**. Várd meg a „Resume to import” jelzést.
6. **Resume**. Az átvett értékkel megnyílik az Inspector.
7. Dolgozz a `capturedInput` változóval, vagy **Freeze DATA**-val készíts stabil bemenetet.

A capture ellenőrzi a cél JVM és a REPL-session azonosságát. Nem folytatja automatikusan az alkalmazást; a függő átadás legfeljebb öt percig érvényes. Ugyanaz az objektum kerül át, nem automatikus másolat. A folytatott program közben módosíthatja, ezért egy változatlan mintát DATA-ként őrizz meg.

Evaluate at Caret vs. Evaluate in frame: előbbi a normál forrásból küld kódot a REPL változóival; utóbbi a megállított metódus lokális változóit látja. Ezért egy forrásbeli `inputDto` nem feltétlenül futtatható közvetlenül a REPL-ben.

16. HTTP: mentett kérés összeállítása

Hely: HTTP fül. Balra a mentett kérések listája, jobbra a részletek, alul a HTTP-konzol látható. A panel saját HTTP-klienssel, az IDE folyamatából küldi a kérést; ehhez önmagában nem kell JShell-kapcsolat.

Mező	Mire való?
Case ID	A kérés azonosítója a panel és a generált helper számára. Nem azonos a snapshot CASE fogalmával.
Név	A kérés emberileg olvasható neve a listában.
Fő téma / Verzió / Leírás	A mentett kéréshez tartozó szervezési és magyarázó metaadat.
HTTP	Metódusválasztó és URL. GET, POST, PUT, PATCH, DELETE, HEAD érhető el.
Headerek	Szerkeszthető Header / Érték sorok. Például Content-Type és Authorization.
JSON Body	A küldendő törzs szövege. A felirat nem jelent automatikus JSON-validálást.

Példa egy fejlesztői kéréshez

1. Új kéréshez használd a bal lista **+ / Add** ikonját.
2. Adj nevet és a saját alkalmazásodhoz tartozó teljes URL-t.
3. Válassz metódust; JSON-hoz adj `Content-Type: application/json` fejlécet.
4. A JSON Body mezőbe illeszd a szükséges bemenetet.
5. **Mentés**, majd külön **Play**. A státuszkód és válasz az alsó konzolban jelenik meg.

A például automatikusan felkínált `http://localhost:8080/api` kezdőérték, nem garantáltan létező endpoint. A helyes címet a saját alkalmazásod route-ja és portja alapján add meg.

Változóhelyettesítés és adattárolás

URL-ben, fejlécben és törzsben `${ENV_NAME}` alakú helyettesítés használható. **Play** esetén az IDEA folyamat környezete számít. A generált snippet JShellben futtatva az alkalmazás JVM-jének környezetét látja. Az IDEA-ból indított alkalmazás Run Configuration környezeti változói ezért nem feltétlenül látszanak a HTTP panelnek.

Az URL, header és body érzékeny részei a panel mentésekor PasswordSafe-ba kerülnek. A munkafüzetbe illesztett literálok viszont az elmentett `.jsh` vagy RECIPE forrásában is benne maradnak.

17. HTTP: gombok és végrehajtás

Az ikonok felirata tooltipként is megjelenhet. A lista **dupla kattintása futtat**; ez eltér a Snapshots vagy CASE szerkesztési munkamenetétől.

Gomb / gesztus	Hatás
+ / Add	Új menthető kérésdefiníciót készít kezdő mezőértékekkel. Nem küld HTTP-kérést.
- / Remove	Megerősítés után törli a kijelölt kérésdefiníciót. Nem HTTP DELETE művelet.
Duplicate	A kijelölt kérés másolatát hozza létre új azonosítóval. Másolás előtt mentsd a módosításaidat.
Run Selected	A kijelölt kérés aktuális mezőivel HTTP-kérést küld.
Lista dupla kattintás	Ugyanazt a futtatást indítja.
Header hozzáadása	Üres fejlécsort szúr be.
Header törlése	A kijelölt fejlécsort törli; kijelölés nélkül az utolsót.
Mentés	Elmenti az űrlap aktuális tartalmát.
Play	Mentés után elindítja az aktuális kérést.
httpReq.perform snippet	Az elmentett kérések helperét és a kiválasztott kérés hívását a Java REPL-be illeszti. Csak az ottani Run küld kérést.

Kérésváltáskor és több más panelműveletnél is menti az aktuális mezőket. Tudatos ellenőrzési pontként használd a Mentés gombot. A snippet a kéréskészletből készül; nézd át a beillesztett kódot futtatás előtt.

A konzol értelmezése

A konzol megmutatja a metódust, a kérés útvonalát, a státuszkódot, időtartamot, Content-Type-ot és a válasz rövidített szövegét. A 2xx státusz HTTP-sikert jelent; a body üzleti jelentését külön ellenőrizd. A HTTP-konzol nem ugyanaz, mint a Java REPL Value / JSON-fa.

Kapcsolódási időkorlát: **10 másodperc**. Kérésidőkorlát: **30 másodperc**. A törzs és a fogadott válasz korlátja **1 MiB**; a konzol ennél rövidebb, legfeljebb 65 536 karakteres body-előnézetet mutat. Ugyanazon kérésazonosító újrafuttatása a korábbi kliensoldali hívás megszakítását kéri, de a szerveroldali hatást nem vonja vissza.

Capture összekötése: előbb Snapshots > Capture next > Arm capture rule, utána az illeszkedő HTTP-kérés Play. Ezután ellenőrizd a capture SAVED állapotát; pusztán a HTTP 200 nem bizonyítja a snapshot elkészültét.

18. AI: promptból Java-javaslat

Hely: AI fül. A plugin saját API-kliensét használja. Ez a panel nem a külső Claude MCP-kapcsolata; az a külön MCP fülön állítható be.

Mező / gomb	Működés
Request	Ide írd, milyen Java-snippetet szeretnél.
Exact prompt sent to the configured API	A teljes, szerkeszthető szöveg, amelyet a küldés használ.
Generated Java code	A kapott válasz; a REPL-be illeszthető.
Refresh bean metadata	Lekéri a beanek neveit és típusait, majd frissíti a prompt alapjául szolgáló metaadatot.
Prepare prompt	A kérésből és a legfeljebb 500 bean metaadatából összeállítja a promptot. Nem küld API-kérést.
Send reviewed prompt	A középső mező aktuális szövegét elküldi a Settingsben beállított API-nak.
Insert response into REPL	A kapott szöveget a munkafüzet kurzorához illeszti, és oda vált. Nem futtatja.

Javasolt sorrend

1. A Settingsben válaszd ki az AI-szolgáltatót, modellt és URL-t; add meg a szükséges API-kulcsot.
2. **Refresh bean metadata**, majd töltsd ki a Request mezőt.
3. **Prepare prompt** után nézd át a teljes középső mezőt. Szükség esetén javítsd.
4. **Send reviewed prompt**. A válasz érkezése után **Insert response into REPL**.
5. A munkafüzetben **Check code**, majd külön **Run cell / selection**.

Ha a Request mezőt vagy a bean-metaadatot utólag módosítod, újra Prepare prompt kell: a Send nem állítja össze automatikusan újra a promptot. A prompt maximum 100 000 karakter.

AI vagy MCP?

AI panel: kódjavaslatot kérsz; te illeszted be és futtatod. A panel megnyitása nem küld adatot a szolgáltatónak.

MCP: a külső Claude-kliens a te feladatod és az engedélyezett eszközök szerint kér állapotot, ellenőriz, futtat, inspectál és snapshotot kezel. Külön saját REPL-sessiont kap.

A Settings **mcp-offline** értéke a beépített AI-panel letiltott módja. A neve ellenére nem indít MCP-szervert; ehhez **MCP > Start MCP** szükséges.

19. Imports és pluginbeállítások

Imports fül

A táblázat oszlopai: **On**, **Alias**, **Fully Qualified Name**. Az **+ / Add** új sort készít, az **Edit** a kijelölt sort szerkeszti, a **- / Remove** törli. Az On jelölő az alkalmazandó importokat választja ki.

Az Add Import / Edit Import ablakban Alias, Fully qualified name és Enabled mező látszik. Az Alias szervezési elnevezés; nem Java `as` alias. A runtime importálás a teljes osztálynevet használja, például `java.util.List`, majd a kódban a `List` névvel hivatkozhatsz rá.

Alkalmazás: Java REPL > Apply configured imports. Az Imports táblázat átírása önmagában nem távolítja el a már futó session importjait. Új bejegyzés létrehozásakor a jelenlegi kód bekapcsolt állapottal ment; szükség esetén utána az On oszlopban kapcsold ki.

Settings / Preferences > Spring Boot Debug REPL and MCP

Mező / kapcsoló	Mire való?
Endpoint (optional; run configs discover their own)	Kézzel választott helyi agent-endpoint. A checkboxos futtatás a sajátját automatikusan megtalálja.
Agent JAR (empty = bundled)	Egyéni agent. Üresen a pluginba csomagolt, hozzá illő verziót használja.
Agent port (0 = allocated by the OS)	Az nREPL agent portja. A 0 szabad portot választ; nem az MCP-port.
Connect the selected endpoint when the project opens	A megadott endpointot projektmegnyitáskor megpróbálja csatlakoztatni.
Persist REPL history (may contain application data)	Az előzmények tartós tárolása. Alapesetben memóriában élnek.
Show results beside source for Evaluate at Caret	Az Evaluate at Caret és Run Selection eredményét a forrás mellett mutatja, Inspect / Snapshot / Output műveletekkel.
AI (mcp-offline = disabled)	A beépített AI-panel szolgáltatómódja.
API key	PasswordSafe-ban tárolt kulcs; támogatott fallback az IDEA környezetének OPENAI_API_KEY értéke.
Model / Base URL	A beépített AI-kliens modell- és végpontbeállítása. A saját szolgáltatódnak megfelelő értéket használd.

A módosításokat az IDEA **Apply / OK** gombjával mentsd. Az agentre vonatkozó beállítások nem cserélik le egy már futó JVM agentjét; új indítás vagy új attach szükséges. A HTTP-portot a Spring-alkalmazás, az agentportot ez a Settings, az MCP-portot az MCP fül kezeli.

20. MCP: a helyi kiszolgáló gombjai

Hely: az utolsó fő fül. Előbb legyen kapcsolat a futó alkalmazáshoz. A munkafüzet MCP gombja csak erre a fülre navigál.

Elem	Funkció
Start MCP	Helyi HTTP-kiszolgálót indít az aktuális REPL-célhoz. Rögzíti a portot és a kiválasztott jogosultságokat.
Stop MCP	Leállítja az elérést és lezárja a klienssessionöket. Az alkalmazás fut tovább.
Port (0 = free)	0: szabad port; más érték: kért helyi port. Futás közben nem szerkeszthető.
Copy client config	Az URL-t és a privát Authorization tokent tartalmazó JSON-t a vágólapra másolja. Futó szerver kell hozzá.
Allow Java execution / state changes	Alapból ki. Fő engedély a végrehajtó és állapotmódosító eszközökhöz. Az olvasási/elemzési eszközök külön is használhatók, a saját kapcsolóiktól és az eszközlistától függően.
Allow HotSwap	Alapból ki. Külön engedély a kódfrissítésre; az előző kapcsoló is kell hozzá.
Allow snapshot / CASE writes	DATA/CASE írás, import és reprodukció létrehozása; külön engedély, alapból ki.
Allow snapshot deletion	Törlés külön engedélye, alapból ki.
Allow CASE execution	Mentett CASE futtatása, alapból ki.
Share IDE recordings with MCP	Az aktuális IDE-felvétel, rögzített érték és forrás megosztása. Alapból ki; olvasáshoz nem kell Java-futtatás. Részletek a 40. fejezetben.
Allow capture / trace changes	Capture módosítása; alapból ki.
Choose allowed tools	Egyenként engedélyezhető eszközök; a kategóriakapcsolók is érvényesek.
Java / CASE mode	ROLLBACK, READ_ONLY vagy LIVE. MCP-ben alapból ROLLBACK. A munkafüzet módjától független.
Transaction manager	Pontos bean-név. Üresen csak az egyetlen elérhető kezelőt választja.
Timeout ms / Calls per session / Result chars	100-120000 ms; 1-100000 hívás; 1024-65536 karakter. Alapértékek: 30000, 1000, 65536.
Redact likely secrets in MCP results	Alapból be; ismert titokminták kitakarása, nem teljes személyesadat-felismerés.
MCP URL	Csak olvasható helyi cím, `/mcp` végződéssel. Ezt add a kliensnek.
Clients / Last tool	Aktív klienssessionök száma és a legutóbb hívott eszköz.

Jogosultságváltás és újraindítás

Stop MCP > kapcsolók beállítása > Start MCP. A projekt megőrzi az utolsó sikeres figyelő portot, a 0-val választott portot is. A projekt tokenjét az IDE PasswordSafe tárolja az IDE jelszótárolási beállításai szerint. A másolt klienskonfiguráció MCP- vagy IDE-újraindítás után is használható. Másik URL-hez válassz 0-t vagy másik portot; URL-változás vagy a mentett hitelesítő adat törlése után másold újra a konfigurációt. Leállításkor a klienssessionök és LIVE referenciák megszűnnek; csatlakoztasd újra az alkalmazást, majd indítsd el az MCP-t.

A szerver csak `127.0.0.1` címen figyel. Legfeljebb 4 MCP-session lehet nyitva; 30 perc inaktivitás után lejárnak. REPL-kapcsolat cseréje/megszakadása és projektbezárás is leállítja az MCP-elérést.

Mit oszt meg a két kliens?

IDE-változók és Claude-változók: külön session, külön handle, külön LIVE pin. **Spring beanek és DATA snapshotok:** ugyanazon alkalmazásban közösek. A külön session nem adatbázis-tranzakció és nem rollback.

A Copy client config által másolt általános JSON-t a kliens formátumához kell igazítani. Claude Code-nál a `"type": "http"` mezőt is meg kell adni; a következő fejezet kész példát mutat.

21. Claude Code: kapcsolódás és első feladat

Claude Code ugyanazon a gépen fusson, amelyen az IDEA MCP-kiszolgálója elérhető. Az alkalmazás projektkönyvtárában add hozzá a szervert, a két helykitöltő cseréjével:

```
claude mcp add --transport http --scope local spring-boot-repl \
  'http://127.0.0.1:PORT/mcp' \
  --header 'Authorization: Bearer GENERATED_TOKEN'
```

A PORT az MCP URL portja; a GENERATED_TOKEN a Copy client config fejlécéből származik. A `Bearer` előtag egyszer szerepeljen. A parancs személyes, ehhez a projekthez tartozó kapcsolatot ad hozzá. Claude-ban a `/mcp` felületen ellenőrizd a státuszt. Új tokenhez a helyi bejegyzés eltávolítható, majd újra hozzáadható:

```
claude mcp remove --scope local spring-boot-repl
```

Ezután az aktuális értékekkel add hozzá újra, és nyiss új Claude Code-sessiont. A tokent ne illeszd beszélgetésbe vagy repositoryba.

Fájlos konfiguráció alternatívaként

A projekt `.mcp.json` fájljában, a meglévő szervereket megtartva:

```
{
  "mcpServers": {
    "spring-boot-repl": {
      "type": "http",
      "url": "${SB_REPL_MCP_URL}",
      "headers": {
        "Authorization": "Bearer ${SB_REPL_MCP_TOKEN}"
      }
    }
  }
}
```

A két környezeti változót a Claude-ot indító terminálban állítsd be. Ez a CLI-s módszer alternatívája; ugyanazt a kapcsolatot egy helyen konfiguráld. Forrás: Claude Code MCP dokumentáció.

Első kérés Claude-nak: „Kérd le a REPL állapotát és a beanek első 50 sorát. A kész contexten ellenőrizd a ctx.getBeanDefinitionCount() kódot futtatás nélkül, majd értékeld ki.”

A repositoryban a `docs/claude-repl-instructions.md` fájlt add át Claude-nak munkautasításként. A részletes kliensútmutató: `docs/claude-repl-guide-hu.md`. A kapcsolat szintaxisát helyi Claude Code 2.1.76 súgóval ellenőriztük; a teljes interaktív kliens próba még nincs igazolva.

22. Claude Desktop és másik gép

Claude Desktop helyi MCP-beállításához a külső **mcp-remote** adapter használható. Node.js/npm szükséges. A rögzített 0.14.2 csomag kapcsolóit a kiadott README alapján ellenőriztük; a teljes Desktop-kapcsolódást nem próbáltuk ki.

macOS konfiguráció: `~/Library/Application Support/Claude/claude_desktop_config.json`. Őrizd meg a meglévő bejegyzéseket, és cseréld a helykitöltőket:

```
{
  "mcpServers": {
    "spring-boot-repl": {
      "command": "npx",
      "args": [
        "-y", "mcp-remote@0.14.2",
        "http://127.0.0.1:PORT/mcp",
        "--allow-http", "--transport", "http-only",
        "--header", "Authorization:${SB_REPL_AUTH_HEADER}"
      ],
      "env": {
        "SB_REPL_AUTH_HEADER": "Bearer GENERATED_TOKEN"
      }
    }
  }
}
```

Indítsd újra Desktopot. Ha nem található az npx, a `command` értéke legyen a terminálban kapott `command -v npx` teljes útvonala. Az adapter az első indításkor letöltődik. Forrás: mcp-remote dokumentáció.

Miért nem működik a localhost a webes connectorban?

A claude.ai, Cowork és a Desktop távoli **Custom connector** kapcsolata az Anthropic felhőjéből indul. Nem látja a géped `127.0.0.1` címét. A Desktop helyi MCP-konfigurációja külön mechanizmus. Ehhez a pluginhoz ugyanazon gépen futó Claude Code-ot vagy a Desktop helyi MCP-kapcsolatát használd. Claude hálózati követelmények.

Másik gép: telepítsd ott a plugin ZIP-et, indítsd ott az alkalmazást és az MCP-t, majd az ottani URL-t/tokent add az ott futó klienshez. A DATA-t az IDE fájlexport/import funkciójával viheted át; a betöltéshez a szükséges DTO-osztályoknak ott is rendelkezésre kell állniuk. LIVE referenciát nem lehet gépek között átadni.

23. Teljes munkamenet: rögzítésből javítás

Ez a példa összeköti a füleket. A `com.example` osztályneveket és a metódust a saját alkalmazásodhoz igazítsd.

1. Szerezz valódi bemenetet

Alkalmazáskódbeli trigger esetén **Snapshots > Capture next > Arm capture rule**, majd küldd el a kérést a HTTP fülről vagy a szokásos kliensből. Várd meg a **SAVED** állapotot. Ha a bemenet csak egy lokális változóban látható, használd a **Debugger > Capture to REPL > Resume** sort, utána Freeze DATA.

2. Töltsd vissza és vizsgáld meg

Snapshots > Saved > Load, változó: `input`. **Variables > input > Inspect**. A Fields nézetben ellenőrizd a mezőket és az elemtípusokat. A bemenet eredetét snapshotnévvel őrizd meg.

3. Készíts rövid kísérleti cellát

```
// %% Service
var processor = ctx.getBean(com.example.CvProcessor.class);

// %% Kísérlet
var actual = processor.process(input);
actual
```

Check code, majd a szükséges cellák külön futtatása. Az `actual` érték Inspect result segítségével megnyitható. Mentsd új DATA-névvel, hogy a kiinduló mintát ne írd felül.

4. Rögzíts elvárt viselkedést

Készíts elvárt DATA-t, majd **Cases / Reload > Save case**. A CASE-ben a service-t a `ctx` segítségével kérd le; a munkafüzet `processor` változója nem kerül át. **Run saved cases** megmutatja a javítás előtti állapotot.

5. Frissíts és ellenőrizz

Módosítsd a Java-forrást. **Use open Java editor > Reload + run selected**. Csak sikeres HotSwap után tekintsd az új futást a módosított kód ellenőrzésének. Strukturális változásnál indítsd újra az alkalmazást, állítsd helyre a bemenetet DATA-ból és futtasd újra a CASE-t.

6. Dolgozz Claude-dal ugyanebből az adatból

Indítsd az MCP-t, és kérd Claude-tól: „Töltsd vissza a cv-input-42 DATA snapshotot input néven; elemezd a releváns mezőket, majd a megbeszélt service-hívással reprodukáld a hibát.” Claude saját sessionjében dolgozik. A teljes 200 MiB-os objektum helyett a szükséges mezőket/lapokat kérje le. A saját eredményét új DATA-néven adhatja vissza az IDE-nek.

24. Gyorsbillentyűk és menük

Az alábbiak a pluginban regisztrált alapértékek. A keymap és az operációs rendszer felülírhatja őket. A munkafüzet billentyűi annak szerkesztőjében működnek.

Művelet	Billentyű és hatókör
Run cell / selection	Cmd+Enter vagy Ctrl+Enter; Java REPL munkafüzet.
Run + next	Shift+Enter; munkafüzet.
Complete	Ctrl+Space; munkafüzet.
Run Selection	Ctrl+Shift+R; normál forrásszerkesztőből kijelölt kód.
Evaluate at Caret	Meta+Shift+E; macOS-en Cmd+Shift+E. A kurzornál lévő Java-kifejezést a REPL-ben értékeli ki.
Reload Class	Meta+Shift+R; macOS-en Cmd+Shift+R. A Java-osztályt fordítja és HotSwapot kér.

Meta macOS-en Cmd. Evaluate at Caret és Reload Class esetén a plugin nem regisztrál külön Ctrl-alapú Windows/Linux változatot. A **Settings / Preferences > Keymap** alatt keress a művelet nevére, és rendelj hozzá szabad kombinációt.

Code > Spring Boot Debug REPL and MCP és a Java helyi menüje

Menüpont	Mit indít?
Run Selection / Evaluate at Caret	Kiértékelés a REPL-sessionben; nem a metódus lokális környezetében.
Advanced Editor	Előhozza a Spring Boot Debug REPL and MCP ablakot; a munkafüzethez válaszd a Java REPL fület.
Sync Imports	A Java-forrás importjainak átvitele a sessionbe.
Trace Method	Az aktuális metódus követése.
Record Class Calls…	Osztályhívások rögzítése: gráf, forrás melletti bemenet és eredmény. Részletek a 39. fejezetben.
Reload Class	Kézi kódfrissítés.
Help (PDF) > English / Magyar	A választott beépített kézikönyv, offline is.

Az IDEA **Tools** menüjében külön szerepel **Attach & Inject Dev Runtime** és **Bind Spring Context**. A 42 workbench-parancshoz saját gyorsbillentyű rendelhető; új alapértelmezett globális kombinációt nem foglalnak le. A Help (PDF) művelethez is adhatsz saját billentyűt.

További workbench-parancsok: a Run, Workspace, Session és Tools menü műveleteire az IDEA Find Action keresőjében `REPL:` névvel kereshetsz, és a Settings > Keymap felületen saját kombinációt állíthatsz be. A forrásbeli **Snapshot point…** a Java-editor helyi menüjében, a margó helyi menüjében és a Code > Spring Boot Debug REPL and MCP menüben is megjelenik.

25. MCP-eszközreferencia Claude-nak

A `tools/list` az engedélyezett eszközöket adja vissza a pontos sémával. Az itt szereplő neveket a kliens szerverprefixszel mutathatja. Paraméter nélküli hívás argumentuma `{}`; ismeretlen kulcs hibát okoz. Az alábbi táblákban a kötelező argumentum félkövér.

A hagyományos listák lapozási alapértéke 50, maximuma 100 sor. A recording eszközöké 20, maximum 50; részletek a 40. fejezetben. A `nextOffset` jelzi a folytatást. A `content` szöveges JSON-t, a `structuredContent` objektumot ad; az `isError` és a runtime hibaállapota ellenőrizendő. CASE esetén az `outcome` mező külön mutatja a próba eredményét.

Ajánlott sorrend: repl_status > szükséges bean/adat lekérése > repl_analyze > repl_eval > meglévő handle inspectálása/mentése. A nem végrehajtó analyze nem hoz létre változót az élő sessionben. Contextcsere után reset, új kapcsolatban DATA-ból visszatöltés kell. Timeout után az üzleti hívást ne játszd újra automatikusan; előbb ellenőrizd a bekövetkezett hatásokat.

Állapot és futtatás nélküli olvasás

Eszköz	Argumentum és cél
repl_status	Nincs; PID, context-ready, context-epoch és képességek.
repl_list_beans	`offset`, `limit`; beanek neve/típusa.
repl_analyze	**code**; Java ellenőrzése végrehajtás nélkül.
repl_complete	**code**, **cursor**; UTF-16 kurzoroffsetes kiegészítés.
repl_variables	`offset`, `limit`; saját sessionváltozók.
repl_imports	Nincs; saját importok.
repl_snapshot_list	`offset`, `limit`; elérhető mentések.
repl_snapshot_info	**name**; metaadat.
repl_snapshot_versions	**name**, `offset`, `limit`; versions-json lista, alap 20, maximum 100.
repl_snapshot_provenance	**name**, `version`; környezet és ellenőrzött SHA-256.
repl_notebook_symbols	**code**; deklarációelemzés Java-futtatás nélkül.
repl_snapshot_diff	**before**, **after**, `offset`, `limit`; DATA eltérések.
repl_snapshot_export	**name**; kis JSON-előnézet, nem garantált teljes export.
repl_capture_status	`rule-id`; saját szabály állapota. Üresen a legutóbbi saját.
repl_capture_list	Nincs; saját szabályok, számlálók és utolsó mentés.
repl_execution_policy	Nincs; runtime/MCP mód, kezelők és maradék MCP-kvóta.
repl_execution_preflight	**code**; mellékhatásokra utaló szöveges jelek, végrehajtás nélkül.

Eszköz	Argumentum és cél
repl_audit_events	Nincs; saját session legutóbbi runtime auditbejegyzései.
repl_events	Nincs; korlátozott eseménylista.
repl_case_list	`offset`, `limit`; CASE-név, tagek, disabled, sorok száma.
repl_case_result	**name**, `row`; saját session utolsó eredménye, újrafuttatás nélkül.
repl_case_export_junit	**name**, `package`, `class`, `file`; fájlmanifeszt vagy egy Java/JSON fájl előnézete, futtatás nélkül.
repl_case_load	**name**; definíció olvasása, futtatás nélkül.

Java és objektumok

Eszköz	Argumentum és cél
repl_eval	**code**; Java a saját tartós sessionben.
repl_interrupt	Nincs; saját futás megszakításának kérése.
repl_reset	Nincs; sessionértékek eldobása, aktuális context használata.
repl_bind_spring	Nincs; indulás után ctx bekötése.
repl_add_imports	**imports**; újsorral elválasztott Java importutasítások.
repl_inspect	**Pontosan egy:** `handle`, `var` vagy `event`.
repl_inspect_page	`offset`; az aktuális objektum másik mezőoldala. Nincs limit paramétere.
repl_inspect_push	**revision**, **index**; a saját legutóbbi inspector-válaszból vett mező megnyitása.
repl_inspect_back	Nincs; előző objektum.

Mentés, trigger, teszteset, kódfrissítés

Eszköz	Argumentum és cél
repl_snapshot_save	**name**, `type`, és **egy**: `handle` / `var` / `event`; DATA.
repl_snapshot_pin	**name**, és **egy**: `handle` / `var` / `event`; LIVE.
repl_snapshot_load	**name**, `var`, `type`, `version`; alap változó: loadedSnapshot.
repl_snapshot_restore_version	**name**, **version**; korábbi tartalomból új aktuális verzió, Java nélkül.

Eszköz	Argumentum és cél
repl_workspace_export	**path**, **state-path**; helyi ZIP írása meglévő workspace JSON-nal. Execution és snapshot-write engedély kell.
repl_workspace_import	**path**, **prefix**; ellenőrzött csomagból új snapshotnevek, mapping-válasz. IDE-workbookot nem nyit meg.
repl_snapshot_delete	**name**; kiválasztott mentés és összes verziójának törlése.
repl_snapshot_import	**name**, **json**; kis JSON-sztring importja.
repl_capture_arm	**point**, **name**, `type`, `case`, `ttl-ms`, `count`, `sample-every`. A case pontos kérésazonosító-szűrő.
repl_capture_disarm	`rule-id`; saját várakozó szabály visszavonása.
repl_events_start	`label`; tap-feliratkozás.
repl_events_stop	Nincs; tap leállítása.
repl_case_save	**name**, **input**, **expected**; `code`, `type`, `variable`, `expected-exception`, `expected-message`, `assertions-json`, `parameters-json`, `result-expression`, `imports`, `setup`, `teardown`, `tags`, `disabled`, `max-duration-ms`, `max-sql-count`, `max-sql-repetitions`, `max-hibernate-loads`, `max-hibernate-flushes`, `max-hibernate-lazy-loads`, `max-hibernate-response-lazy-loads`. Code vagy result-expression szükséges.
repl_case_run_batch	**names**; új sorral elválasztott CASE-nevek, közös időkeret.
repl_case_run	**name**; mentett eset futtatása.
repl_reproduction_create	**name**, **input**, `variable`, `type`, `metadata-json`; utolsó futásból csomag, újrafuttatás nélkül.
repl_reload	**code**; teljes Java-forrás, külön HotSwap-engedéllyel. Nincs path/file/className argumentum.

IDE-felvételek és gráf (0.20)

Az alábbi 17 eszköz a közös IDE-felvételt kezeli; a katalógus összesen 82 eszközt tartalmaz. **Share IDE recordings with MCP** szükséges. A részletes lapozási szabályok és egy Claude-munkamenet a 40. fejezetben található.

Eszköz	Argumentum és cél
repl_recording_status	Nincs; az IDE aktuális felvételének azonosítója, view verziója, állapota és letöltött hívásszáma.

Eszköz	Argumentum és cél
repl_recording_calls	**recording**, `view`, `query`, `errors-only`, `root`, `focus`, `thread`, `min-duration-ms`, `from-ms`, `to-ms`, `collapsed`, `offset`, `limit`; szűrt hívásfa és hívóút.
repl_recording_call	**recording**, **call**, `view`; pontos overload, breadcrumb, gyerekek, previous/next/nextError és forráselérhetőség.
repl_recording_values	**recording**, **call**, **part**, `view`, `path`, `offset`, `limit`, `text-offset`, `text-limit`; input/result/exception rögzített mezői.
repl_recording_source	**recording**, **call**, `view`, `offset`, `limit`; rögzített forrás lapozva, fájlnév, eredeti SHA-256 és metódussor.
repl_recording_timeline	**recording**, `view` és a calls szűrői, `offset`, `limit`; szálankénti időadatok. Nincs collapsed argumentum.
repl_recording_pin	**recording**, **call**, `view`; egy letöltött hívás rögzítése a kliens saját összehasonlítási referenciájaként.
repl_recording_compare	**recording**, **after**, valamint **egy**: `before` / `reference`; opcionális `view`, `offset`, `limit`. Mezőszintű eltérések és részlegesség.
repl_recording_select	**recording**, **call**; a node kijelölése az IDE-ben, forrás és rögzített értékek megnyitása. Állapotmódosítási engedély kell.
repl_recording_start	**expected**, **classes**; `sql` (sztring true/false, alapból true), `hibernate` (true/false, alapból sql), `n-plus-one-threshold` (2-1000, alapból 5); az aktuális felvétel azonosítója vagy none, 1-8 különböző pontos osztálynév új sorral elválasztva. Capture/trace és állapotmódosítási engedély is kell.
repl_recording_stop	**recording**; az adott megosztott felvétel leállítása. Capture/trace és állapotmódosítási engedély is kell.
repl_recording_hibernate	**recording**; `view`, `root`, `call`, `event-id`, `kind`, `offset`, `limit`. Hibernate 6.6 metaadatok, session, entity, kapcsolat, lazy/flush/cache események és kapcsolódó SQL-azonosítók.
repl_recording_hibernate_findings	**recording**; `view`, `root`, `call`, `offset`, `limit`. SELECT-ekkel összekötött lazy N+1-gyanú és válaszkészítés közbeni lazy betöltések.
repl_recording_hibernate_compare	**recording**, **after**, és pontosan egy: `before` / `reference`; `view`. Hibernate-számlálók előtte/utána, különbség és részlegesség.
repl_recording_sql	**recording**; `view`, `root`, `call` (teljes híváság), `sql-id`, `offset`, `limit`, `text-offset`, `text-limit`. JDBC-események és SQL-szöveg lapozva; szülő/root, idő, datasource, forrás és hibafajta.
repl_recording_findings	**recording**; `view`, `root`, `offset`, `limit`. Kérésenkénti N+1-gyanús SELECT-csoportok, darabszám, idő és példaazonosítók.
repl_recording_sql_compare	**recording**, **after** és pontosan egy: `before` / `reference`; opcionális `view`. Két híváság SQL-statisztikája és eltérése.

26. Hibaelhárítás a felületről

Jelenség	Mit ellenőrizz?
Not connected / nincs READY	Az alkalmazás indítási logját, a REPL-checkboxot és a Java REPL státuszát. Spring-hibát a Bind nem javít meg.
Nincs aktív profil	Active profiles mező vagy `--spring.profiles.active=...`; ne puszta vesszős argumentum legyen.
Nem látszik a checkbox / régi classloader-hiba	A friss ZIP-et telepítsd, majd indítsd újra az IDEA-t. Ellenőrizd a támogatott IDE-verziót és a régi egyéni agent beállítást.
A lokális változó ismeretlen	Evaluate at Caret REPL-scope-ban fut. Lokális változóhoz debugger frame vagy capture kell.
Az Imports átírása után még nincs típus	Apply configured imports; a táblázat mentése nem alkalmazza automatikusan az importokat.
Context changed	Reset, majd DATA betöltése és a szükséges előkészítő cellák kézi futtatása.
Partial preview / Partial code check	Csökkentsd a vizsgált adatot vagy kódot. Inspector Fields-ben nyiss kisebb részt.
Capture nem készül el	Pontos pontnév, case ID, bridge, élesítés és valóban bekövetkezett alkalmazáshívás. Debugger capture után Resume kell.
Snapshot betöltése sikertelen	Legyen elérhető a DTO és Jackson; ellenőrizd a mentett generikus típust. CASE-t a Cases / Reload lapon nyiss meg.
Reload elutasítva	Fordítási hiba vagy JVM által nem támogatott szerkezeti változás. Utóbbinál alkalmazás-újraindítás kell.
HTTP-ből hiányzik környezeti változó	A Play az IDE környezetét látja, a snippet az alkalmazását. Ellenőrizd, hol állítottad be.
MCP 401 / kapcsolat elutasítva	Csatlakoztatott alkalmazás és Start MCP; ellenőrizd a mentett portot, a PasswordSafe-et és a pontos Authorization fejlécet. URL-változás vagy a mentett hitelesítő adat törlése után másold újra a konfigurációt.
Claude Code command hibát jelez	JSON-konfigurációban legyen `"type": "http"`. Az URL önmagában kevés.
MCP GET /mcp hibát ad	Az SSE-hez bearer token, inicializált session és Accept: text/event-stream kell. Nem nyilvános weboldal; lásd a 49. fejezetet.
MCP-eszköz hiányzik	A jogosultságokat leállított MCP mellett módosítsd, utána Start és új klienskonfiguráció.
Variables Previous/Next nem lapoz	Az Inspector saját Previous/Next gombjait használd.

Bizonytalan kimenetel: timeout vagy kapcsolatvesztés után ne ismételj automatikusan író hívást. A Reset nem undo; az Interrupt csak megszakítást kér. A rollback határai: 28. fejezet.

27. Korlátok, fogalmak és további anyagok

A fontos korlátok egy helyen

Terület	Jelenlegi korlát / működés
DATA snapshot	Alapból 200 MiB, metaadatokkal. Nem heapkorlát.
Inline JSON	IDE: 2 MiB; MCP import: 100 000 karakter, a kéréskorlát mellett.
MCP átvitel	Kérés 256 KiB; válasz protokollplafon 512 KiB, külön 1024-65536 karakteres beállított korlát. Nagy DATA a runtime-ban marad.
MCP session	Legfeljebb 4; 30 perc inaktivitás; sessionönként 4096 külön kérésazonosító.
LIVE / esemény	LIVE legfeljebb 30 perc; tap/trace 128 érték és öt perc/session.
Capture szabály	JVM-enként 16 aktív; 1-100 mentés/szabály. UI-ban 5 perc; runtime-ban 1 ms és 30 perc közötti TTL.
Objektumelőnézet	500 érték, 6 szint, 50 gyermek/konténer, összesen 131 072 szövegkarakter.
Live check	100 000 forráskarakter; 200 deklaráció/100 000 kontextuskarakter; 200 snippet; 100 diagnosztika.
Ellenőrzési időkeret	5 másodperc, fordítói műveletek között ellenőrizve; egy javac-műveletet nem állít le erőszakkal.
HTTP	10 s kapcsolódás, 30 s kérés; 1 MiB törzs/válasz; korlátozott konzolelőnézet.

A csonkolás és részleges vizsgálat jelzését mindig vedd figyelembe. Egy nulla eltéréses, de részleges compare nem bizonyít egyenlőséget.

Fogalmak

Session: egy tartós Java-kiértékelési munkatér. **ctx:** a Spring context. **Handle:** sessionhöz kötött azonosító egy meglévő eredményhez. **DATA:** szerializált érték. **LIVE:** élő referencia. **RECIPE:** mentett kód. **CASE:** bemenet + elvárt eredmény + futtatandó kód. **HotSwap:** támogatott kódmódosítás átvétele a futó JVM-ben.

Források és frissítés

A kézikönyv a repository `src/main/kotlin/hu/baader/repl/ui`, `ai`, `mcp`, `settings` és `runner` kódjára, a menüregisztrációra és a runtime műveleteire épül. A nem bekötött régi paneleket nem sorolja aktív funkcióként.

Szerkeszthető forrás: `docs/repl-help-hu.md` és `docs/repl-help-en.md`; a `scripts/build-help-pdf.py` alapból mindkét PDF-et újraépíti. A Claude-nak átadható részletes eszközleírás: `docs/claude-repl-instructions.md`. A kliensbeállítások bővebben: `docs/claude-repl-guide-hu.md`. A 0.22 ellenőrzése: `docs/history/HIBERNATE_0_22.md`; SQL-felvételek: `docs/history/SQL_RECORDINGS_0_21.md`; korábbi IDE-kompatibilitás: `docs/history/IDEA_2025_2_0_13_1.md`.

A Help (PDF) a pluginba csomagolt példányt nyitja meg. Az új kézikönyv a friss csomag telepítésével kerül az IDE-be; önállóan ebből a PDF-ből is használható. A felület működését a forrás alapján ellenőriztük; ez nem jelenti minden üzleti alkalmazás vagy Claude-kliens teljes körű tesztelését.

28. Futtatási mód és tranzakció

A Java REPL eszköztára alatt az alkalmazás neve, profiljai, kapcsolata és a szerver által visszaigazolt futtatási mód látszik. A beállítás az IDE-session Java- és CASE-futtatására vonatkozik. A natív session alapból LIVE; az MCP külön szabályt használ, alapból tiltott Java-futtatással és ROLLBACK móddal.

Elem	Használat
LIVE / DB rollback / DB read-only	A választás azonnal beállítási kérést küld. A kijelző a szerver visszaigazolása után vált át. Nincs külön Apply gomb.
Applying / Reading execution settings	A beállítás még folyamatban van. Java és CASE nem indítható sem gombbal, sem gyorsbillentyűvel.
Mode unconfirmed	A tényleges állapot nem ismert. Session > Refresh execution settings kéri le újra. Sikertelen váltás után automatikus visszaolvasás is történik.
Session > Execution settings	Tranzakciókezelő bean neve és időkeret egy ablakban. Több kezelő esetén válassz konkrétat. Alap időkeret 30000 ms; 100-120000 ms állítható be.
Session > Refresh execution settings	A tényleges módot és az elérhető kezelőket olvassa vissza.
Tools > Side-effect hints	A kijelölt vagy teljes forrásban HTTP-re, üzenetre, fájlra, folyamatra vagy aszinkron munkára utaló jeleket keres, futtatás nélkül.
Tools > Audit events	A saját session utolsó runtime auditbejegyzéseit írja a konzolba.

Futás alatt a módváltás letiltott. A tooltip megmutatja a teljes alkalmazásnevet, profilokat és PID-et, ha rendelkezésre állnak. A profiljelzés tájékoztatás: profilváltáshoz a Run Configuration beállítását módosítsd, és indítsd újra az alkalmazást.

ROLLBACK/READ_ONLY esetén új, szinkron tranzakció indul a Java futtatás szálán, az input visszatöltése és a CASE ellenőrzése is ezen belül történik. A kezelő rollbacket kap siker, Java-hiba és visszatért megszakítás után is. Hiányzó vagy többértelmű kezelő esetén a futtatás el sem indul; nincs csendes visszaváltás LIVE-ra.

A garancia határa: csak a kiválasztott kezelő tranzakciójában részt vevő szinkron DB-munka görgethető vissza. Belső REQUIRES_NEW, másik kezelő, async/reactive munka, HTTP, Kafka/RabbitMQ, fájlírás és memóriabeli beanváltozás kívül esik ezen. A read-only driver/kezelő számára adott jelzés, nem általános írástiltás. Nincs általános undo, outbound firewall vagy automatikus mock.

A timeout együttműködő megszakítást kér; nem garantálja egy megszakítást figyelmen kívül hagyó hívás megállását. A tranzakció létrehozása előtt nincs aktív Java-időzítő. A konzol és MCP-válasz jelzi a módot, a rollback eredményét és az eltelt időt. Ismeretlen kimenetelt ne tekints sikernek és ne ismételj automatikusan író hívást.

A tranzakciós szemantikát a Spring programozott tranzakciókezelése adja: docs.spring.io/spring-framework/reference/data-access/transaction/programmatic.html .

29. Reprodukció egy csomagban

Hely: Java REPL > Tools menü: **Create reproduction**, **Export reproduction**, **Import reproduction**. A művelet egy már kiválasztott DATA bemenetből és az utolsó Java-futtatás kódjából/eredményéből készül; nem rekonstruálja visszamenőleg egy tetszőleges alkalmazáshívás állapotát.

1. Mentsd a hibás bemenetet Capture vagy Saved segítségével DATA-ként.
2. Töltsd vissza például `input` néven. Futtasd az ezt használó reprodukáló cellát. A cella legyen önálló a kiválasztott bemenettel és importokkal; más korábbi sessionváltozókat nem ment a csomag.
3. Kattints **Create reproduction**. Adj nevet, válaszd a bemeneti DATA-t, a kódban használt változónevet és szükség esetén típust. Opcionálisan megadhatsz tenant/feature flag/HTTP metaadatot.
4. A plugin új nevű input- és expected DATA-t, CASE-et, RECIPE-et és környezeti DATA-t hoz létre. Utána exportálható a `.sbrepl-bundle` fájl. A kód eközben nem fut újra.
5. A másik környezetben **Import reproduction**. Az import új neveket használ, majd a Cases / Reload fülön külön, tudatosan futtathatod az esetet.

A Create párbeszédablak az utolsó mentett HTTP-kérést külön jelölőnégyzettel tudja a metaadatokhoz adni. Ez nem bizonyítja, hogy a kérés okozta az utolsó REPL-futtatást. A kérés nem indul el és importkor nem kerül automatikusan az HTTP fülre.

Környezeti bizonyíték: időpont, Java-verzió, PID, context epoch, szál, locale/timezone, aktív/default profilok, elérhető Spring Boot-, build- és Git-adatok. Az input eredeti capture-környezetét is megőrzi. Git/build információ csak akkor lesz, ha az alkalmazás közzéteszi propertyként vagy erőforrásként. A SecurityContext nevét csak az aktuális szálon elérhető Spring Security kontextusból ismeri.

Nem állítja vissza: principal, jogosultságok, tenant, clock, profilok, feature flag-ek, külső szolgáltatások és élő objektumok. Az utolsó eredményt a Create pillanatában másolja, ezért közben módosult objektumnál nem az eredeti futáskori állapotot kapod. A bemeneti DATA eredeti másolata viszont elkülönül az azóta felülírt forrástól.

A ZIP-manifeszt szerepet, méretet és SHA-256 ellenőrzőösszeget tartalmaz. Importkor csak az ismert hat fájl fogadható el, útvonalbejárás és duplikáció tiltott, a kibontott összméret legfeljebb 200 MiB. A checksum sérülést jelez; nem digitális aláírás és nem hitelesíti a küldőt.

30. Hibát elváró CASE és capture-metaadat

A Cases / Reload definíciós részében két új mező van: **Expected exception** és **Expected message**. Üres exception esetén az eredmény a korábbi strukturális DATA-összehasonlítással ellenőrződik. Kitöltve pontosan a kivételosztály és az üzenet egyezését várja. Nem subclass- vagy regex-illesztés. Fordítási hiba nem válik elvárt üzleti kivétellé.

A runtime kivétellel végződő utolsó futásból készített reprodukció automatikusan ilyen CASE-et hoz létre. Ez a korábbi hibát rögzíti; a javítás után változtasd meg az elvárt viselkedést, különben a régi hibát váró teszt továbbra is azt kéri számon. Az expected DATA mindig kötelező, exception esetén a csomag hibaleírást ment bele.

Tenant és feature flag rögzítéséhez az alkalmazásszálon használd a bridge új overloadját:

```
SnapshotHelper.captureLazy(
    "order-processing", requestId, () -> inputDto,
    java.util.Map.of("tenant", tenantId,
                     "feature.newFlow", "true"));
```

Legfeljebb 32 szöveges metaadatmező, kulcsonként 128, értékenként 2048 karakter adható. A felismert titoknevű mezőket kitakarja. Ezek bizonyítékok, nem automatikusan újraaktivált thread-local értékek.

Több felvételhez: **Capture count = 5**, név `order-${sequence}`, opcionális pontos case ID, majd **Arm capture rule**. A saját szabálytábla mutatja a számlálót és az utolsó snapshotot. A szabályoknak nem lehet átfedő tervezett snapshotnevük. Már létező, inaktív mentés azonos néven új verziót kap. A tartós verziókat a Snapshots > Versions mutatja.

31. Audit, titkok és további fejlesztések

A runtime és az MCP privát helyi JSONL naplót ír a **~/.java-repl-audit** könyvtárba. Külön fájl tartozik a runtime és az IDE PID-jéhez. 4 MiB felett rotál; az aktuális fájl és öt korábbi példány marad. A könyvtár jogosultsága POSIX-rendszeren 700, a fájlé 600.

A naplóban szerepel a session/kliens, művelet, kezdés és lezárás, időtartam, kód és kódhash, illetve a runtime-ban context epoch. A capture mentése külön naplózott művelet. CASE-nél a ténylegesen betöltött forrás kerül a naplóba. Kezdő audit nélkül nem indul művelet; lezárási audit hibájánál a válasz jelzi, hogy hatás már történhetett. Az Audit events a saját session aktuális fájlbeli legfeljebb 100 bejegyzésének korlátozott előnézete.

A felismert jelszó-, token-, Authorization-, cookie- és hasonló mintákat kitakarja az auditból, a historyból és alapbeállítás mellett az MCP-válaszokból. Az AI-panel küldés előtt ugyanilyen találat esetén módosítja a prompt előnézetét, majd újabb áttekintést és Send kattintást kér. A detektor heurisztikus: nem talál meg minden titkot vagy személyes adatot. Az eredeti workbook, DATA/RECIPE tartalma, transcript és a csomag üzleti adatai nem automatikusan anonimizáltak. Export előtt nézd át őket.

Az MCP engedélyei eszközszintű kapcsolók; a tetszőleges Java-futtatás nem sandbox. Engedélyezett Java-kód ugyanabban a JVM-ben van, és elérheti az alkalmazás osztályait, beanjeit. A Java-kódra nincs megbízható package-, bean- vagy metódus-allowlist. A sessionkvóta új sessionnel újrakezdődik; a napló nem manipulációbiztos és nem korlátlan megőrzésű.

A 0.14-ben elkészült: tranzakciós futtatási mód, mellékhatás-jelzés, reprodukció létrehozás/export/import, finomabb MCP-kapcsolók, audit, több capture szabály és kivételt elváró CASE.

A 0.15-ben elkészült: mezőszintű assertionök, paraméterezett CASE, setup/cleanup, JUnit-export és a hozzájuk tartozó MCP-eszközök. Részletek a következő fejezetekben.

A későbbi fejezetekben már elérhető: notebookállapot, workspace-mentés, snapshot-verziók, forrásbeli snapshotpont, hívásgráf és idővonal, SQL/N+1 és Hibernate-megfigyelés (35-42. fejezet).

További lehetséges fejlesztések: automatikus snapshot-sémamigráció, kimenő hívások tiltása vagy stubolása, általános reactive követés, távoli konténer/Kubernetes kapcsolat, önálló headless/CI runner, titkosítás és csapatszintű registry. Ezeket a kiadás nem állítja működő funkciónak.

32. CASE 2.0: mezők és assertionök

A **Cases / Reload** fülön a korábbi CASE-ek továbbra is futnak. A **Load selected** megnyitja a mentett definíciót. A **Save case** elmenti a mezőket, de nem futtat kódot. **Run saved cases** a mentett változatot futtatja; a szerkesztett szöveget előbb mentsd.

Mező vagy alfül	Mire való?
Code	Java-utasítások. Régi CASE-ben az utolsó JShell-érték adja az eredményt.
Result expression	Opcionális külön Java-kifejezés, például input.size(). A Code után fut, ez lesz az eredmény. JUnit-exporthoz kötelező; ne tegyél ide pontosvesszőt.
Assertions	JSON-beállítások. Üresen vagy {} esetén teljes expected DATA-egyezés.
Parameters	Több mentett input/expected pár ugyanahhoz a kódhoz. Üres mezőnél a felső DATA-választók érvényesek.
Setup / cleanup	Importok, előkészítő Java-kód és lezáró Java-kód külön mezőkben.
Options	Vesszővel elválasztott tagek, Disabled jelölés, Maximum code duration milliszekundumban.
Export JUnit ZIP	Egy kijelölt, elmentett CASE-ből Java-tesztet és JSON-erőforrásokat készít. Nem futtatja az esetet.

Az assertionök JSON Pointer útvonalakat használnak: az üres sztring a gyökér, **/items/0/name** az első elem neve. A **/items/*/id** egy szinten minden elem id mezőjét jelöli. Kulcsban a / jelet ~1, a ~ jelet ~0 alakban írd. A * foglalt wildcard; ez nem teljes JSONPath nyelv.

```
{
  "ignore": ["/id", "/items/*/createdAt"],
  "unordered": ["/items"],
  "numericTolerance": {"/total": 0.01},
  "timeToleranceMs": {"/updatedAt": 1000}
}
```

include: csak a megadott útvonalak részfáit hasonlítja. **ignore:** kihagyja a megadott részfákat. **unordered:** az adott listában sorrendtől független, egy-egy elemhez rendelt összehasonlítás; a duplikációk számítanak. Legfeljebb 200 elemű rendezetlen lista támogatott. Toleranciánál nem mohó párosítást használ, így az átfedő tartományok sem okoznak hamis eltérést.

numericTolerance: abszolút, nemnegatív numerikus eltérés; sztringet nem alakít számmá. **timeToleranceMs:** ISO instant vagy offsetet tartalmazó dátum/idő sztringek eltérése. Offset nélküli LocalDateTime nem támogatott ennél az assertionnél. Pontos útvonal-beállítás megelőzi a wildcardot; több illeszkedő wildcardnál az első szerepel.

```
{
  "compareSnapshot": false,
  "checks": [
    {"path":"/items", "op":"hasSize", "value":2},
    {"path":"/items/*/code", "op":"matches",
     "value":"[A-Z][0-9]+", "match":"all"},
    {"path":"/items/*/active", "op":"equals",
     "value":true, "match":"any"}
  ]
}
```

A check operátorai: **equals**, **contains** (szövegrész vagy listaelem), **hasSize** (lista/map/sztring), **matches** (teljes regex-egyezés), **exists**, **notNull**, **isNull**. A match alapértéke all; any esetén egy találat elég. Hiányzó útvonal és üres wildcard-találat sikertelen. Az explicit checket az ignore nem kapcsolja ki. A compareSnapshot=false legalább egy checket igényel; expected DATA-t ilyenkor is válassz.

Legfeljebb 100 útvonal/check, 64 KiB JSON-beállítás és 200 000 összehasonlítási lépés használható. Mélység-, regex- vagy munkakorlátnál nem lesz hamis PASSED: INCONCLUSIVE, vagy már bizonyított eltérés esetén FAILED az eredmény. Az ismeretlen opció és hibás JSON már mentéskor elutasításra kerül.

33. Paraméterezés, életciklus és eredmény

A Parameters mező JSON-tömböt vár, 1-20 különböző id-val. Minden sor létező DATA snapshotneveket használ:

```
[
  {"id":"small", "input":"input-small",
   "expected":"expected-small"},
  {"id":"large", "input":"input-large",
   "expected":"expected-large"}
]
```

Egy futás sorrendje: új session és tranzakcióhatár, input betöltése, importok, setup, Code, Result expression, assertion, cleanup, majd rollback a kiválasztott mód szerint. A következő sor új inputpéldánnyal és új tranzakcióval indul. A workbook változói nem kerülnek át, a Spring beanek és a LIVE mellékhatások közösek.

A **Maximum code duration** 1-120000 ms közötti utólagos assertion. A runtime-ban a Code és az eredménykifejezés kiértékelését méri, a JShell fordításával együtt; nem tartalmazza az inputbetöltést, setupot, JSON-összehasonlítást és cleanupot. Nem megszakítási időzítő. A felső futtatási szabály külön, kooperatív határideje továbbra is érvényes, és a CASE összes sorára közös keret jut.

A **Disabled** CASE SKIPPED eredményt ad kódfuttatás nélkül. A tagek megőrződnek, MCP-n listázhatók és JUnit @Tag annotációként exportálódnak. Mappák és tagalapú futtatásszűrés még nincs.

Setup-hiba nem teljesíthet expected exception assertiont. **Cleanup** normál eredmény, assertion- vagy futási hiba után is megkísérelt; megszakítás után kimarad. Cleanup-hibával az eset ERROR lesz. Az adatbázis-tranzakció lezárása ettől függetlenül a futtatási szabály feladata. Az alkalmazáskód által figyelmen kívül hagyott megszakítás nem garantál azonnali leállást.

A sorok egymás után futnak. FAILED után a többi sor folytatható; ERROR és CANCELLED leállítja a hátralévő sorokat. Az összesített állapot mellett a riportban soronkénti állapot, idő, eltérés, output és rollback-információ jelenik meg. A részletes riport korlátozott előnézet; az Inspect actual egyetlen sor esetén használható közvetlenül, több sor esetén az Events nézetben válaszd ki a megfelelő CASE-eseményt.

MCP-ben: **repl_case_list**, **repl_case_run_batch**, **repl_case_result**. A batch names mező új sorral elválasztott 1-20 CASE-nevet fogad, összesen legfeljebb 100 paramétersorral. Az összes mentett definíció és DATA-hivatkozás ellenőrzése megelőzi a futást. Külön CASE-jogosultság kell; a kliens nem írhatja felül az MCP-módot vagy határidőt.

A repl_case_result a saját session legutóbbi eredményét olvassa, nem indít futást. A row argumentum 0-tól induló index. Legfeljebb 20 CASE eredményét tartja meg; reset/contextváltás után ezek lejárnak. A batch és a CASE hívást timeout után ne ismételd meg automatikusan.

34. CASE-ből JUnit 5 teszt

1. A Cases / Reload fülön válassz input- és expected DATA-t. Az **Optional input type** mezőbe exporthoz kötelező a teljes Java-típus, például java.util.List<java.lang.Integer>.
2. A Code mezőbe Java-utasításokat írj; a végső értéket helyezd a Result expression mezőbe. Például Code: input.add(1); és Result expression: input.size(). Az importokat a külön Imports mezőbe tedd.
3. Mentsd a CASE-et, jelöld ki a táblában, majd **Export JUnit ZIP**. Adj Java-package-et, tesztosztálynevet, futtatási módot és szükség esetén tranzakciókezelőt. Az export párbeszédablak alapmódja ROLLBACK.
4. A ZIP src/test/java és src/test/resources tartalmát másold az alkalmazás megfelelő tesztkönyvtáraiba. Nézd át a JSON-adatokat, tesztprofilokat és külső függőségeket.
5. Futtasd az alkalmazás szokásos buildjével, például ./mvnw -Dtest=ReproductionTest test vagy ./gradlew test --tests 'reproduction.ReproductionTest'.

A csomag tartalma: @SpringBootTest + @ParameterizedTest osztály, ApplicationContext injection ctx néven, input/expected JSON minden sorhoz, assertions.json, a runtime-mal azonos forrású hordozható assertion-segéd és README. Az eredményeket AssertJ ellenőrzi. Java 17+, spring-boot-starter-test, Jackson és spring-tx szükséges. A normál Maven/Gradle tesztfutás adja a JUnit XML riportot; IDE és REPL-agent nem kell.

Az export csak Java-szintaxist ellenőriz. Az alkalmazás típusait és tesztfüggőségeit a projekt buildje ellenőrzi. A JShell-specifikus deklarációkat, agent-segédeket és korábbi sessionaliasokat előbb alakítsd normál Javává. Saját Jackson codec/mixin esetén a generált mapper testreszabható. A runtime aktív profilját vagy titkait nem aktiválja automatikusan a tesztben.

A generált teszt is külön tranzakciót és rollbacket használ soronként a kiválasztott mód szerint. Több kezelőnél konkrét név kell. A cleanup a bemenetet és ctx-t éri el; az exercise metódus lokális változóit nem. A futásidő-assertion JUnitban a kódot méri, JShell-fordítás nélkül, ezért nem pontosan azonos teljesítménymérés. A megszakítás az eredeti végrehajtási szálon történik, így nem választja le a tranzakciót a kódról.

Az export méretkorlátja összesen 16 MiB. Nagy snapshotból készíts kisebb reprodukciós fixture-t. A korábbi .sbrepl-bundle v1 formátum egy rögzített input/expected párt hordoz; paraméterezett vagy más DATA-ra átkötött CASE esetén JUnit ZIP-et használj. A notebook/workspace mentést és a snapshot-verziózást a 35-37. fejezet mutatja be; a külön bundle CLI még fejlesztési terv.

Claude: repl_case_export_junit(name, package, class) fájlmanifesztet ad. Ugyanez file argumentummal egy kiválasztott forrást/JSON-t ad vissza, fájlírás és futtatás nélkül. Az inline korlát 64 KiB fájlonként, erre az MCP eredménykorlátja és redactionje is rájön. Csonkolt vagy kitakart válaszból ne állíts össze éles tesztfixture-t; a teljes csomaghoz az IDE ZIP-exportja használható.

A JUnit paraméterezés és timeout működését a JUnit 5 dokumentációja írja le. A tranzakciós korlátokhoz lásd a Spring teszttranzakciók és a 28. fejezet leírását.

35. Notebook: cellaállapot és függőségek

Hely: Java REPL munkafüzet és Cells eredményfül; a sorozatfuttatások a Run menüben vannak. A korábbi `.jsh` fájlok és `// %%` cellahatárok tovább használhatók.

Eredmény közvetlenül a cellánál: minden cella alatt megjelenik a futási sorszám, állapot, időtartam és rövid kimenet. Az elavulás jelzése megnevezi a módosult bemeneti cellát is, amikor az ismert. A bal margón levő futtatásjelre kattintva ugyanitt érhetők el a műveletek.

Cellaművelet	Hatás
Run	Ezt az egész cellát futtatja. A korábbi kijelölést megszünteti.
Inspect	Az adott futás élő eredményét nyitja meg az Inspectorban. Nem futtatja újra a kifejezést.
Snapshot	A meglévő eredményből DATA-t ment. Egy ablakban megadható a név, meglévő cél és opcionális Java-típus. Azonos név új verziót hoz létre.
Output	A Cells lapon megnyitja az adott cella tárolt kimenetét.

Nagyon keskeny szerkesztőben a Run és Output link látszik, a többi jobb kattintással elérhető. Reset vagy újracsatlakozás után a korábbi élő eredmény Inspect/Snapshot művelete letiltódik. A forrás módosítása után a megjelölt régi futás eredménye még megvizsgálható, amíg az élő referenciája érvényes. Az objektum későbbi mutációját csak a már elmentett DATA fagyasztja be.

Forrásfájlban, CIDER-mintára: az Evaluate at Caret és Run Selection eredménye a kód mellett marad, Inspect / Snapshot / Output linkekkel. Az Output nagyítható strukturált nézetet nyit. A beállítás az Imports/Settings fejezetben található. Ez továbbra is a REPL-sessionben futtat; a metódus lokális változóját debuggerből kell átvenni, vagy a 38. fejezet szerinti snapshotponttal rögzíteni.

Jelzés / mező	Jelentés
Run [12]	A cella legutóbbi futásának sorszáma. Minden tényleges cellafuttatás új sorszámot kap.
SUCCESS / ERROR / INTERRUPTED / NEVER	Siker, hiba, megszakadt állapot vagy még nem futtatott cella. A siker mellett is lehet stale figyelmeztetés.
modified	A jelenlegi forrás eltér attól, amelyhez a mentett eredmény tartozik.
stale	Bemenet, session, cellasorrend vagy külső állapotváltozás miatt a korábbi eredmény elavult lehet.
Last run / ms	Indulási idő és a legutóbbi végrehajtás időtartama.
Depends on	A függő cellák számai. Az alsó nézet a deklarált neveket, lehetséges bemeneteket és a cella saját kimenetét mutatja.
Go to cell	A kijelölt sor forrásához viszi a kurzort.

Gomb	Működés
Analyze dependencies	Deklarációkat elemez JShelllel, futtatás nélkül. Nem telepíti a deklarációkat az élő sessionbe.
Run above	A kurzor cellája előtti nem üres cellákat futtatja, forrássorrendben.

Gomb	Működés
Run from here	A kurzor cellájától a munkafüzet végéig futtat.
Run affected	Újraelemzi a függőségeket; a kijelölt cellát és ismert függőit futtatja, forrássorrendben.
Restart + run all	Megerősítés után reseteli a sessiont, majd minden nem üres cellát sorban futtat.
Run all	Sorban futtatja a cellákat, session-reset nélkül.

A sorozat az első hibánál, szerkesztésnél, kapcsolatváltozásnál vagy Interrupt után leáll. A már futó cella hatásait ez nem vonja vissza. A kijelölés külön snippet: attól a teljes cella nem minősül lefutottnak. Külső editorból futtatás, változóbetöltés és HotSwap után a korábbi cellabizonyíték elavultnak jelölhető.

A gráf korlátja: deklarációk és lehetséges Java-azonosítóhivatkozások alapján készül. Metódushívások, utasítások, tömbök és nem elemzett cellák esetén korábbi cellákat is függőségnek tekint. Aliasok, reflection és külső beanállapot teljes követése nincs; nincs automatikus topologikus átrendezés vagy ciklusfeloldás. A függőségjelzés nem bizonyítja, hogy egy üzleti hatás biztonságosan ismételhető.

Automatikus checkpoint: a munkafüzet, a legutóbbi cellaeredmények és a futtatási sorrend helyi, projektenként elkülönített fájlba kerül az IDEA rendszerkönyvtárán belül. Legfeljebb 200 cella, cellánként 16 384 karakter kimenet, 1000 futási bejegyzés és 8 MiB workspace-metaadat. A checkpoint forrást és alkalmazásadatot tartalmazhat; nem a VCS-be kerül. Újranyitáskor a korábbi eredmények látszanak, de stale jelzést kapnak: a Java-objektumok nem éledtek újra.

36. Teljes workspace mentése és megnyitása

Hely: Java REPL > Workspace menü; a Save workspace külön ikont is kapott. A `.sbrepl-workspace` egy ellenőrzőösszegekkel ellátott ZIP, nem futtatható csomag. Az **Open workbook** továbbra is csak `.jsh`/szöveg megnyitása.

Gomb	Mit tartalmaz vagy végez?
Save workspace	Munkafüzet, cellaazonosítók, eredmények, futási sorrend, importok, DATA-eredetkötések, Inspector-bookmarkok, választható HTTP-kérések és transcript. Kapcsolat esetén az aktuális tartós DATA / CASE / RECIPE fájlokat és a DATA-kötések szükséges régi verzióit is csomagolja.
Open workspace	Ellenőrzi a csomagot, majd jóváhagyás után lecseréli a szerkesztő munkafüzetét. Kapcsolat esetén új prefix alá importálja a snapshotokat; a CASE input/expected és paramétersor-hivatkozásai átíródnak. A HTTP-kérések külön azonosítóval kerülnek be.
Workspace info	Mentett környezet, DATA-kötések, bookmarkok és DATA-eredet nélkül maradt változók listája.
Restore DATA binding	Egy mentett DATA-verziót explicit kiválasztás után materializál a megadott Java-változóba. DTO-konstruktor futhat, létező változó lecserélődhet.
Insert saved imports	A mentett importokat a szerkesztőbe illeszti. Külön Run szükséges a végrehajtáshoz.

Javasolt munkafolyamat: mentsd el a szükséges értékeket DATA-ként, töltsd be őket névvel, majd Save workspace. Másik sessionben Open workspace, Workspace info, Restore DATA binding; végül csak a szükséges előkészítő és kísérleti cellákat futtasd. A forrásba kézzel beírt snapshotnevek/RECIPE-kódok nem írhatók át általánosan: az import új neveihez ezeket szükség esetén igazítsd.

A mentett DATA-kötés egy fagyasztott eredetre mutat. Az objektum azóta megváltozhatott: a csomag nem állítja, hogy minden élő változó az export pillanatában pontosan a snapshot értékét tartalmazta. Az egyéb Java-objektumokat, LIVE pinjeit, handle-jeit, beanállapotot, adatbázist és külső szolgáltatásokat nem menti vagy állítja vissza. Profil, principal, tenant, időzóna és verzióadat tájékoztató; import nem módosítja az alkalmazás környezetét.

Inspector-bookmark: named változóból induló megnyitás után Bookmark menti az aktuális útvonalat. Open bookmark az aktuális sessionben keresi meg; a gyökérváltozót előbb állítsd vissza. Hiányzó, kétértelmű vagy 10 000 gyermek fölött nem ellenőrizhető útvonalnál megáll. Remove bookmark törli a kiválasztott bejegyzést. Result/handle gyökérnél előbb Bind current, majd nyisd meg a változót a Variables fülről.

Korlátok: 200 MiB összes tömörítetlen adat és legfeljebb 200 MiB ZIP; 8 MiB metaadat; legfeljebb 1200 csomagolt snapshotbejegyzés. Egy 200 MiB-os snapshot mellé a metaadat már nem fér el ebben a csomagban: ilyenkor használj külön snapshot-exportot vagy kisebb projekciót. Az import előbb minden objektumot előkészít és ellenőriz, majd ütközés nélkül ír új nevekre; a meglévő snapshotokat nem cseréli le. Ez nem folyamatösszeomlást is átfogó többfájlos adatbázis-tranzakció.

Kapcsolat nélkül a Save workspace csak a helyi munkaterületet menti, alkalmazás-snapshotokat nem. Open workspace offline is megnyitja a forrást; a DATA-fájlokhoz kapcsolódj és nyisd meg újra a csomagot. A teljes export/import azonos fájlrendszert igényel az IDEA és a JVM között. HTTP-kéréseknél és választható transcriptnél a felismert titkok kitakarásra kerülnek; a forrás, cellakimenet és DATA hű másolat, export előtt ellenőrizendő. A csomag nem titkosított.

37. Snapshot-verziók és származási adatok

Hely: Snapshots > Saved: jelölj ki egy tartós mentést, majd **Versions**. LIVE-hoz nincs lemezre mentett verziótörténet.

Gomb / oszlop	Funkció
SHA-256 / Current	A teljes envelope tartalmi lenyomata és jelzés, hogy ez-e az aktuális verzió.
Recorded / Kind / Bytes	Rögzítési idő, DATA / CASE / RECIPE típus és fájlméret.
Provenance	A kiválasztott verzió ellenőrzött lenyomata, eredeti capture ideje, Java/Spring környezet, aktív profilok, context epoch és elérhető Git/build adatok.
Load version	Egy régi DATA-verziót új változóba tölt; az aktuális snapshotot nem változtatja meg.
Restore as latest	A korábbi tartalomból új aktuális verziót készít, restoredFrom hivatkozással. A jelenlegi és a régi fájl is megmarad. Java-kódot nem futtat.
Refresh	Újraolvassa a történetet. A lista előnézet; a teljes SHA-ellenőrzés megnyitáskor/visszaállításkor történik.

Az új mentések privát, tartalom alapján címzett fájlba kerülnek. A korábbi v1 aktuális fájlformátum megmaradt; első újramentéskor a régi érték is archiválódik. Egy névhez legfeljebb 100 verzió tartozhat. Nincs automatikus előzménytörlés: a korlátnál a mentés hibával megáll, a régi adat megmarad. Válassz új nevet, vagy exportálás után tudatosan töröld a teljes régi snapshotot. A **Saved > Delete** az aktuális fájlt és minden verzióját törli.

A származási adatok az eredeti capture-hez tartoznak; workspace-import külön sourceApplicationId/importedFrom, visszaállítás külön restoredFrom/recordedAt adatot ad. A Java mezőséma SHA-256 lenyomata a gyökértípus reflektált mezőit jelzi; nem teljes Jackson-séma, és nem migrál DTO-kat. Az ellenőrzőösszeg sérülést érzékelhet, digitális aláírást vagy titkosítást nem helyettesít. Automatikus sémamigráció, régi DTO-osztály visszatelepítése és teljes történetű csapatregistry nincs.

Claude új eszközei: **repl_snapshot_versions**, **repl_snapshot_provenance**, **repl_snapshot_restore_version**, **repl_notebook_symbols**, **repl_workspace_export**, **repl_workspace_import**. A teljes referencia a 25. fejezetben található. Workspace exportnál a state-path meglévő workspace JSON-ra mutat a JVM gépén; az MCP nem gyűjti össze helyette az IDEA szerkesztőjét. Az import visszaadja a név- és verzióleképezéseket, a natív UI munkafüzetét nem cseréli le.

38. Snapshotpont egérrel a Java-forrásban

Cél: az alkalmazás következő hívásakor egy kiválasztott sor előtt menteni a helyi változót DATA-ként, alkalmazáskódba írt helperhívás nélkül. A pont az IDEA Java-debuggeréhez tartozik; **Debug + Enable Spring Boot Debug REPL and MCP** szükséges.

1. Indítsd a szokásos Spring Boot konfigurációt Debug módban, bekapcsolt REPL-checkboxszal.
2. Válassz olyan végrehajtható Java-sort, amely előtt a mentendő változó már inicializálva van. Például az `order` létrehozását követő sort.
3. Jelöld ki az `order` kifejezést, majd jobb kattintás > Spring Boot Debug REPL and MCP > **Snapshot point…**. A bal margó helyi menüjéből is indítható; ilyenkor ellenőrizd a kifejezést.
4. Add meg a snapshot nevét, például `order-input`. A **Java expression** a kiválasztott sor lokális környezetében értékelődik ki. A Java-típus opcionális; generikus listánál megadható a teljes típus. A **Capture attempts** alapból 1, maximum 100.
5. Mentés után lila snapshotjel látszik a margón. Válts az alkalmazásra, és indítsd el a vizsgált kérést.
6. A debugger a sor végrehajtása előtt rögzíti az értéket, majd folytatja az alkalmazást. A Debug konzolba siker vagy hiba kerül. A mentést a **Snapshots > Saved** fülön találod; innen töltsd be a REPL-be.

A pont kezelése

A jel helyi menüjében a **Configure snapshot / rearm…** nyitja a beállítást. Ugyanazon a soron ismét a Snapshot point műveletet választva is szerkeszthető. A **Save and rearm** újraélesíti a számlálót. A **Remove point** a pontot törli; a már elmentett DATA megmarad. A natív breakpoint kapcsolójával a pont átmenetileg letiltható.

A pont az IDE projektjének breakpointbeállításában mentődik, és a forrássor mozgatását követi. Másik JVM-induláskor új számláló indul. Egy projekthez legfeljebb 64 ilyen pont hozható létre. A `.sbrepl-workspace` nem exportálja a debugger breakpointjait. Ha ugyanott már hagyományos breakpoint van, válassz másik sort vagy távolítsd el azt.

Mikor nem készül mentés?

Normál Run módban nincs debuggeres sorérzékelés. Inicializálás előtti változó vagy hibás kifejezés nem értékelhető ki. A mentési kísérlet szerializációs hiba esetén is elfogy; javítás után élesítsd újra. A korábbi DATA ilyen hiba miatt nem cserélődik le. A Debug konzol adja a részletes hibát.

A pillanatfelvétel a találat szálán készül. A debugger rövid időre felfüggeszti a szálat a kiértékelés és szerializáció idejére, de siker esetén nem vár kézi Resume-ra. Nagy objektum lassíthatja a kérést; továbbra is a 200 MiB alapkorlát érvényes. Egyszerű változót vagy mezőt válassz, mert egy metódushívásnak vagy egyedi szerializálónak saját mellékhatása lehet. Más szál által módosított objektumról ez nem garantál atomi képet.

Claude/MCP: a kész DATA-t a meglévő snapshot-listázó, info, betöltő és Inspector eszközökkel használhatja. A pont létrehozása és a Java-debugger irányítása ehhez nem kapott új MCP-eszközt.

39. Rögzített hívásfa és értékek a forrás mellett

Cél: a futó alkalmazás egy kiválasztott Java-osztályában látni, melyik metódus milyen bemenettel és eredménnyel futott le. A gráf egy korábbi node-ját kiválasztva megnyílik az osztály forrása a rögzített értékekkel. Ehhez **Run + Enable Spring Boot Debug REPL and MCP** is elég; debugger nem szükséges.

Első felvétel

1. Indítsd a Spring Boot alkalmazást a REPL-checkboxszal, és várd meg a kapcsolódást.
2. Nyisd meg a vizsgálandó Java-osztályt. Jobb kattintás > Spring Boot Debug REPL and MCP > **Record Class Calls…**. A Code menüből és a Find Action keresőből is elérhető.
3. A párbeszédablak előre kitölti az osztály teljes nevét. Megadhatsz további osztályokat, soronként egyet, összesen legfeljebb nyolcat. Csak a megadott osztályok deklarált, konkrét metódusai kerülnek a felvételbe.
4. Indítsd el a rögzítést, majd használd az alkalmazást: például küldd el a vizsgált HTTP-kérést. A felvétel a tényleges hívásokat figyeli, nem indítja el helyetted az üzleti műveletet.
5. Nyisd meg a **Tap / Trace > Recorded calls** alfület. A gráf node-jain látszik a hívásszám, az osztály/metódus, a siker vagy hiba, az időtartam, a rövid eredmény és a szál.
6. Kattints egy node-ra. A megfelelő metódusnál megjelenik a **CALL #…** blokk. Az **Input / result** link nagyítható ablakot nyit; a **Call graph** visszavisz a felvételhez. A gráfon a fel/le nyíl és az Enter is használható.
7. A **Stop recording** leállítja az új hívások rögzítését. A már futó, rögzített hívások befejezése még megérkezhet. A **Save recording…** elmenti a felvételt későbbi visszanézéshez.

Például a feldolgozó és a parser osztály együttes kiválasztásakor láthatóvá válhat a `process` alatti `parse` hívás. Ha csak a feldolgozót választod ki, a parser belseje nem kerül automatikusan a fába. Ugyanannak a metódusnak az ismételt vagy rekurzív hívásai külön node-ok. A túlterheléseket a tényleges paramétertípusok azonosítják.

Gombok és nézetek

Vezérlő	Mire való?
New recording…	Új felvétel az osztályok megadásával. A korábbi felvételt előbb mentsd el, ha meg akarod tartani.
Stop recording	Leállítja a rögzítést; az alkalmazás tovább futhat.
Root	Egy belépő hívás és leszármazottai, vagy az összes belépő hívás megjelenítése.
Follow latest	Az újonnan letöltött hívásokat követi. Node kézi kiválasztásakor kikapcsol, így nyugodtan vizsgálhatod a múltbeli hívást.
Values in source	Az új rögzített eredmények megjelenítése a megnyitott forrásban. Kézi node-kiválasztáskor az érték külön is megnyitható.
Clear source values	Eltávolítja a forrás melletti blokkokat. A felvétel megmarad; a további automatikus blokkok elrejtéséhez kapcsold ki a Values in source opciót.
Open source	Újra megnyitja a kiválasztott node forrását a rögzített bemenettel és eredménnyel.
Input at entry	A metódusba belépéskor rögzített argumentumok. Paraméternév nélkül `arg0`, `arg1` stb. jelenik meg.
Result at exit	A kilépéskor rögzített eredmény. A void és a kivétel miatti visszatérés külön jelzést kap.
Exception	Kivétel típusa és rögzíthető üzenete. Egyedi, felülírt üzenet-előállító kódot nem hív meg a rögzítő.
Tree / Formatted / Raw	Kinyitható mezőfa, formázott és nyers nézet. Expand / Collapse a rögzített fa kibontása és összecsukása.
Inspect live	Az esemény még elérhető élő objektumát vizsgálja. Ez közben változhatott; a rögzített nézet a korábbi értékeket mutatja.
Save recording…	Forrás, gráf és rögzített értékek mentése `.sbrepl-recording` fájlba.
Open recording…	Mentett felvétel megnyitása alkalmazáskapcsolat nélkül is.

A nézet keskeny ablakban egymás alá rendezi a gráfot és az adatokat. A szokásos élek az azonos szálon megfigyelt hívásokat kapcsolják össze. Bekapcsolt async felvételnél a támogatott executorok feladathatára köti össze a beküldőt a munkaszállal (47. fejezet); a nem követett munka külön gyökeret kap.

Gráfnavigáció és szűrés (0.19)

A **Call graph** kártyái a metódusnevet, a státuszt, az időtartamot, a névvel ellátott bemenetet és a rögzített eredményt is mutatják. A státusz szövege mellett külön jel áll: siker, hiba, futó vagy befejezetlen hívás. A **PARTIAL PREVIEW**, **VALUES UNAVAILABLE** és **INCOMPLETE** jelzések megkülönböztetik a részleges, még nem elérhető és befejezetlen adatokat. Rámutatáskor megjelenik a teljes metódusszignatúra, a paraméternevek és egy nagyobb értékelőnézet. A teljes rögzített mezőfa továbbra is a **Values** oldalon nyitható ki.

Vezérlő	Mire való?
+ / mínusz, százalék	Nagyítás és kicsinyítés. Ctrl vagy Cmd + görgő a mutató körüli pontot tartja helyben. A sima görgő függőlegesen, Shift + görgő vízszintesen görget.
Húzás	A gráf bal vagy középső egérgombbal mozgatható. Húzás végén nem választ ki véletlenül node-ot.
Fit graph	Az összes jelenleg látható kártyát az ablakba igazítja. Nagy fán a szöveg kicsi lesz; a Show selected ismét olvasható méretre hozza a kiválasztott hívást.
Show selected	Láthatóvá teszi a kiválasztott hívást és kinyitja a hozzá vezető ágakat. Ha a szűrés elrejti, törli a szűrőket.
+ / mínusz a node sarkában	Egy ág kibontása vagy összecsukása. Összecsukva látszik a rejtett hívások és hibák száma.
Collapse all / Expand all	A hívásfa ágainak összecsukása vagy kibontása. Aktív szűrőnél a találathoz vezető ágak automatikusan nyitva maradnak.
Search	Keresés osztályra, metódusra, paraméternévre és a rögzített értékek szövegére. Kis- és nagybetűtől független, részszöveges keresés. A még le nem töltött vagy nem rögzített értékekben nem keres.
Thread / Errors only / Min ms	Szűrés szálra, hibás hívásra és minimális rögzített időtartamra. A szűrők együtt működnek.
Focus branch	Csak a kijelölt hívás ágát és a hozzá vezető hívókat mutatja.
Clear filters	Törli a keresést, a gyökér-, szál-, hiba-, időtartam-, időszak- és ágszűrést.
Detach window…	Ugyanezt a böngészőt külön, átméretezhető ablakban nyitja meg. Az ablak bezárásával vagy a Return to tool window gombbal visszakerül a panelre; a felvétel megmarad.

A szűrés nem vágja el a hívási utat. A találat előtti hívókat szaggatott keret és **caller path** szöveg jelöli. A találatszám a tényleges találatokat számolja, a megtartott hívókat nem. A keresés, szűrés és a gráf kézi nagyítása, görgetése vagy mozgatása kikapcsolja a Follow latest módot. Frissítéskor így megmarad a kézzel beállított nézet. A követés a kapcsolóval újraindítható; másik felvétel megnyitásakor új nézet indul.

Léptetés a rögzített hívások között

A **Previous**, **Next** és **Next error** az aktuális szűrésnek megfelelő hívások között lépked. Az összecsukott ágban levő találatot is megnyitja. A **Caller** a rögzített közvetlen hívóra ugrik. A sorrendet a belépéskor kiosztott hívásazonosító adja; az utolsó hívás után nincs körbefordulás.

A gombok alatti kattintható útvonal például `#1 Controller.handle > #2 Service.process > #3 Parser.parse`. Bármelyik elem megnyitja az adott hívás forrását és értékeit. A gráf, a Values, a Compare calls és a forrás melletti blokk ugyanazt a kiválasztott hívást követi.

Billentyű a böngészőben	Művelet
Fel / le a gráfon	Előző vagy következő látható kártya.

Billentyű a böngészőben	Művelet
Bal / jobb a gráfon	Ág összecsukása, hívóra lépés, illetve kibontás.
Enter a gráfon	A kiválasztott hívás forrásának megnyitása.
= / mínusz a gráfon	Nagyítás / kicsinyítés.
Home a gráfon	Fit graph.
Alt + bal / jobb	Previous / Next a böngészőn belül.
Alt + le / fel	Next error / Caller a böngészőn belül.

Ezek a billentyűk a gráf vagy a böngésző fókuszában működnek. A forrásszerkesztő és a REPL saját gyorsbillentyűit nem módosítják. A léptetés a rögzített hívásokat járja be; üzleti kódot nem indít.

Referenciahívás és mezőnkénti összehasonlítás

Válassz egy már befejezett és letöltött hívást, majd kattints a **Pin reference** gombra. Ezután válassz egy másik node-ot. A **Compare calls** oldalon a rögzített bemenet, eredmény, kivétel és státusz különbségei jelennek meg. A referencia a kijelölés váltásakor is ugyanaz a rögzített hívás marad. A **Repin reference** lecseréli, a **Clear reference** feloldja.

A táblázat mezőutat, változást, referenciaértéket és kijelölt értéket mutat. Egy sor kiválasztásakor alatta nagyobb szövegben olvasható az eltérés. Az **ADDED**, **REMOVED**, **CHANGED** és **UNKNOWN** szöveget háttérszín is kiegészíti. JSON-string esetén a mezőket strukturálisan hasonlítja össze; tömböknél az index és sorrend számít. Az ismétlődő JSON-mezőneveket külön előfordulásként tartja meg.

Részleges, hiányzó vagy ciklikus referencia-előnézetnél az összehasonlítás jelzi a bizonytalanságot. A nem rögzített adatot nem tekinti biztosan hozzáadott vagy törölt mezőnek. Azonos megjelenítési mezők nem bizonyítják a teljes élő objektumok egyezését. Legfeljebb 2000 eltéréssor és mezőértékenként 8192 karakter jelenik meg; további eltérés esetén a nézet jelzi, hogy az összehasonlítás nem teljes.

Más metódus vagy túlterhelés is összevethető, de ezt a fejléc külön jelzi. A referencia és a kiválasztott hívás időtartama is látható; ezek tájékoztató, instrumentált futási idők. Új felvétel megnyitása törli a helyi referenciakijelölést.

Idővonal és időszak kijelölése

A **Timeline** fül szálanként, időarányos sávokkal mutatja a hívásokat. Az egymásba ágyazott és átfedő hívások külön sorokra kerülnek. Egy sávra kattintva ugyanúgy megnyílik a forrás és a rögzített adat, mint a hívásfa node-jánál.

Az idővonalon vízszintesen húzva kijelölhetsz egy időszakot. A hívásfa az időszakkal átfedő hívásokat és hívóikat tartja meg; az idővonalon a többi hívás halvány marad tájékozódásként. A **Clear time range** csak az időszakot törli, a többi szűrő megmarad. A **Clear filters** az összes szűrést feloldja.

Az időtengely a felvétel legkorábbi rögzített kezdőidejétől számított milliszekundum. A kezdőidő milliszekundum pontosságú rendszeróra, az időtartam külön mért eltelt idő. Az órakorrekció és a rögzítés költsége befolyásolhatja a képet. Futó vagy befejezetlen hívásnál nincs teljes befejezési idő; a rövid jel csak a rögzített részt mutatja. A támogatott szálváltásokat csak bekapcsolt async felvétel kapcsolja össze (47. fejezet).

Mit őriz meg a felvétel?

A bemenet megjelenítési adatai belépéskor, az eredmény adatai kilépéskor másolódnak le. Egy lista későbbi módosítása így nem változtatja meg a korábbi felvételt. A böngésző a már rögzített mezőket bontja ki. A felvétel megnyitása nem futtatja újra a metódust, nem állítja vissza a JVM stackjét, és nem helyettesíti a típusos DATA snapshotot.

A felvétel indításakor a plugin a megtalálható Java-forrást is elmenti. Ha azóta szerkesztetted, a node a mentett változatot nyitja meg **csak olvasható** szerkesztőben. A munkapéldány tartalmát nem cseréli le. A forrás és a futó bájtkód egyezését ez nem bizonyítja: felvétel előtt fordítsd és szükség esetén töltsd újra a módosított osztályt. HotSwap után indíts új felvételt.

Ha a forrás vagy az adott túlterhelés nem oldható fel, a panel ezt jelzi, a rögzített adatok továbbra is megvizsgálhatók. Szerkesztés után a már látható forrásblokkon külön jelzés mutatja, hogy az a korábbi kódhoz tartozik.

Tárolás: sessionönként egy aktuális felvétel, legfeljebb **200 hívás**, összesen **32 MiB megjelenítési adat**. A korlát elérése leállítja a további rögzítést; a korábbi szülő-node-ok megmaradnak. A jelzett kihagyás a korlát elérésére utal, nem teljes számláló minden későbbi alkalmazáshívásról. Legfeljebb 64 egymásba ágyazott megfigyelt hívásszint, osztályonként 64 konkrét metódusnév és az összes sessionben együtt 16 instrumentált osztály támogatott.

Értékelőnézet: egy értékfánál legfeljebb 500 node, hat mélységszint és node-onként 50 gyermek kerül előnézetbe; legfeljebb az első 32 argumentum. A szövegkeret 131072 karakter, egy mélyebben levő string legfeljebb 4096 karakter. A korlátot elérő részen jelzés látszik. A teljes élő objektumot az Inspectorban vizsgálhatod, amíg elérhető, vagy célzott DATA-t menthetsz róla. A DATA 200 MiB-os kerete ettől különálló.

Az alapértelmezett megjelenítési felvétel mezőket olvas, alkalmazásbeli gettert és egyedi szerializálót nem indít. A külön engedélyezett Capture replay DATA viszont snapshot-szerializálást végez (43. fejezet). A hívó szálon végez munkát, ezért lassíthatja a vizsgált hívást; ez nem pontos teljesítményprofilozó. Más szál által módosított objektumról nem garantál atomi képet. Először kevés, célzott osztályt válassz.

Offline fájl: legfeljebb 64 MiB; a mentett forrás fájlonként egymillió, összesen négymillió karakter. A fájl alkalmazásadatot és forrást tartalmazhat, titkosítás nélkül. Betöltéskor ellenőrzi a formátumot, a szülőkapcsolatokat, a méretkorlátokat és a forrás ellenőrzőösszegét. Ez sérülést érzékelhet, hitelességet nem bizonyít. Mentéskor még futó vagy megszakadt kapcsolat miatt befejezetlen hívás **INCOMPLETE** állapotot kap. Kapcsolatvesztés után csak a már letöltött adatok őrizhetők meg. A felvételt külön kell menteni, a workspace-export nem tartalmazza.

Sorértékek: ez a funkció metódusok bemenetét, eredményét és hívásrendjét mutatja. Egy tetszőleges belső sor lokális változójához a 38. fejezet szerinti snapshotpont használható Debug módban.

Claude/MCP: a 0.20-as kiadás 11 új eszközzel megosztja az aktuális hívásgráfot, értékeket, forrást és összehasonlítást. Beállítás és használat a következő fejezetben.

40. AI-agentek hozzáférése a hívásgráfhoz

Minden recording eszközhöz szükséges az **MCP > Share IDE recordings with MCP** kapcsoló; alapból ki van kapcsolva. Olvasáshoz a Java-futtatást nem kell engedélyezni. A **Choose allowed tools** tovább szűkítheti az elérést, például a forrás megosztását. A pin csak a kliens saját rögzített előnézetét módosítja. A select az IDE-kijelölést is módosítja; ehhez a fő állapotmódosítási kapcsoló szükséges. Start/stop esetén a capture/trace kapcsoló is kell.

A 17 eszköz teljes argumentumtáblázata a **25. fejezetben** található. Itt a közös jogosultságokat, az adatolvasás sorrendjét és az összehasonlítás menetét mutatjuk be.

Az alábbi szabályok a felvételi eszközökre vonatkoznak:

- Először `repl_recording_status`. Ha `available=false`, még nincs megosztható felvétel. Indításhoz ekkor `expected="none"`; egyébként az aktuális `recording` azonosító szükséges. Aktív felvételt előbb állíts le. A start a runtime visszaigazolását várja meg; az értékek letöltése ezután folytatódhat.
- Az olvasási válasz egyetlen EDT-állapotból készül; `scope="ide-recording"`. A gráf és az IDE-ben megnyitott mentett felvétel közös a kliensekkel, a Java-session továbbra is különálló. A `call` azonosító nem használható `event`, `handle` vagy `var` helyett.
- A `recording` és a lapozáskor visszaadott `view` együtt őrzi az olvasott állapotot. Ha közben hívás fejeződik be vagy előnézet töltődik le, a régi view elutasítható; az olvasást új view-val, 0. oldaltól kezdd. Felvételcsere után a régi azonosítóval sem kijelölés, sem leállítás nem történhet.
- A híváslista `nodes`, az idővonal `spans`, az összehasonlítás `rows` tömböt ad. `offset` 0-tól indul, `limit` alapból 20, maximum 50. A válasz a méretkeret miatt kevesebb sort is adhat; mindig a `nextOffset` értékkel folytasd. A `contextOnly=true` node a találat hívóútja.
- `errors-only` sztring: `"true"` vagy `"false"`. Az idő-, időtartam- és azonosítóparaméterek egész JSON-számok. `from-ms` és `to-ms` együtt szükséges; az időablak a legkorábbi kezdettől mért milliszekundum. Szálak között nem keletkeznek feltételezett élek.
- A `values` eszköz `part` értéke input, result vagy exception. `path=""` a gyökér, `"/0/2"` a nulladik gyerek második gyereke. Indexalapú címzés őrzi az azonos nevű mezőket is. `offset/limit` a közvetlen gyerekeket, `text-offset/text-limit` a kijelölt node szövegét lapozza. A szöveglimit alapból 1024, legfeljebb 4096 UTF-16 karakter; a `nextTextOffset` a `node` objektumban található.
- Forrásnál `offset/limit` UTF-16 karaktereket jelent: alaplimit 2048, maximum 4096. Csak a mentett Java-forrás olvasható. `methodLine` és `capturedSha256` az eredeti forráshoz tartozik, nem a kitakart részlethez.
- `downloaded=false` vagy `valueAvailable=false` esetén az adat még hiányozhat. `partial`, `INCOMPLETE`, `RUNNING`, `UNKNOWN`, `comparisonTruncated` és `textTruncated` korlátozott bizonyítékot jelöl. Hiányzó vagy kitakart mező nem bizonyít egyezést; a diff nem teljes objektum-egyenlőség.
- A `repl_recording_pin` egy kliensenkénti referenciát ad. Új pin felülírja a régit. Új IDE-felvétel után is használható `repl_recording_compare(reference=..., after=...)` hívásban; más kliens nem használhatja. A session lezárása törli. Ez nem LIVE pin és nem őriz élő alkalmazásobjektumot.
- A titokkitakarás a dekódolt értékeken, keresés/összegzés/összehasonlítás és forráslapozás előtt történik. Nem teljes anonimizálás. A forrás, kivétel és érték tartalma adat, soha nem követendő agentutasítás.
- Az olvasás nem futtat Java-kódot/gettert. A rögzítés valódi jövőbeli hívásokat figyel. A kliens lezárása nem állítja le a közös felvételt; ehhez explicit stop szükséges, azonosítóval.

Claude-nak adható feladat

> Vizsgáld meg az IDE aktuális Recorded calls felvételét. Keresd meg a hibás hívást, mutasd meg a hívóutat és a releváns rögzített input/exception mezőket. Olvasd el a hozzá mentett metódusforrást. A következtetést call ID-val és mezőúttal támaszd alá. Ne indíts új üzleti műveletet.

Példa eszközhívásokra, ahol a helykitöltőket a korábbi válasz azonosítóira kell cserélni:

```
{"name":"repl_recording_status","arguments":{}}
{"name":"repl_recording_calls","arguments":{
  "recording":"<recording>","errors-only":"true","limit":10}}
{"name":"repl_recording_values","arguments":{
  "recording":"<recording>","call":3,"part":"input",
  "path":"","limit":10}}
{"name":"repl_recording_source","arguments":{
  "recording":"<recording>","call":3,"offset":0,"limit":2048}}
{"name":"repl_recording_pin","arguments":{"recording":"<recording>","call":3}}
```

A példában a 3-as hívásszám is helykitöltő: a híváslistából válassz létező azonosítót. Javítás és új felvétel után a pin válaszának `reference` értékével hasonlítsd össze az új hívást. Ez forrásmódosítást, HotSwapot és üzleti újrafuttatást nem végez magától.

Kapcsolat és offline fájlok

MCP-indításhoz élő REPL-kapcsolat kell; annak megszakadása leállítja az MCP-t. A közben IDE-ben megnyitott offline felvétel megosztható. Fájlnyitás/mentés és ablakkezelés az IDE-ben marad. A resource-feliratkozást és az opcionális task-kezelést a 49. fejezet ismerteti.

41. SQL, N+1-jelzés és DB-ig követhető hívások

A 0.21-es kiadás a Recorded calls felvételhez JDBC-megfigyeléseket kapcsol. Szinkron Spring MVC-kérésnél a kérés gyökérhívása, a kiválasztott alkalmazásosztályok metódusai és a tényleges JDBC-műveletek egy fában látszanak. Nem kell hozzá alkalmazáskódba írt SQL-logolás vagy külön adatbázis-plugin.

Indítás és értelmezés

1. Tap / Trace > Recorded calls > **New recording...**. Add meg a fontos controller/service/repository osztályokat (1-8).
2. Hagyd bekapcsolva a **Record JDBC/SQL and synchronous Spring MVC requests** opciót. A **Suspected N+1 repetition threshold** alapértéke 5, tartománya 2-1000.
3. Futtasd le az alkalmazásban a vizsgált HTTP-kérést vagy belépő metódust. A rögzítés önmagában nem indít üzleti kérést.
4. **Stop recording**, majd a **Root** mezőben válaszd a vizsgált kérést. A párhuzamos kérések és a különálló belépő hívások SQL-jei külön gyökérhez tartoznak.
5. Nyisd meg az **SQL & N+1** fület. A csoportok kinyithatók; minden végrehajtásnak saját SQL-azonosítója és valódi szülőhívása van.

A JDBC-node megmutatja az SQL-sablont, a végrehajtások számát, a kliensoldali időt és a datasource osztály/instance azonosítóját. A kapcsolat megszerzésének ideje külön mérés, nem része az SQL-végrehajtások összegének. A teljes Java-hívás ideje a JDBC-munka mellett minden más alkalmazásmunkát és a megfigyelés költségét is tartalmazza.

Nem kiválasztott Java-metódusokhoz nem készülnek kitalált node-ok. A 47. fejezet külön bekapcsolható Executor/@Async/CompletableFuture-követést ad; Reactor, R2DBC, adatbázison belüli tervek és lockok továbbra sincsenek követve. A JDBC-idő kliensoldali mérés; a szinkron megfigyelés maga is lassítja a futást.

Gombok és nézetek

Vezérlő	Működés
Group SQL	A gráfban azonos szülőhöz, SQL-sablonhoz, datasource-hoz és hívási helyhez tartozó műveleteket összevonja. Kikapcsolva egyes végrehajtások látszanak.
SQL & N+1	SQL-részletező: darabszám, idő, datasource, forrásosztály/metódus/sor, szál, root és hibafajta.
SQL / datasource / source + Filter	A rögzített SQL-sablonokban és metaadatokban keres. Enterrel is alkalmazható.
Suspected N+1 only	Csak a küszöböt elérő ismétlődő SELECT-csoportokat mutatja.
Open originating call and source	A kijelölt SQL-esemény rögzített alkalmazáshívójára lép, és annak forrását/értékeit nyitja meg.
Pin SQL baseline	Az aktuális Root-szűrő SQL-adatait rögzíti. Ez a referencia új felvétel vagy offline fájl megnyitása után is megmarad.
Clear baseline	Törli az SQL-referenciát. A Java-értékek Pin reference összehasonlítása ettől külön funkció.
Timeline	Az egyes JDBC-műveletek valódi időszakaszai látszanak. A gráf csoportosítása nem készít belőlük mesterséges folytonos időszakaszt.
Save / Open recording	Az SQL-napló és a beállított küszöb is a felvételbe kerül. A régi fájlok SQL nélkül továbbra is olvashatók.

Mit jelent az N+1-gyanú?

A jelzés egy kérésen/rooton belül az azonos normalizált SELECT, datasource és hívási hely ismétlődéseit keresi. Öt vagy több végrehajtásnál alapból **suspected N+1** jelzés jelenik meg. A hívóág node-jai is figyelmeztető jelzést és SQL-darabszámot kapnak.

Példa: egy Hibernate-kérés öt rendelést olvas, majd minden rendelés vevőjét külön tölti be. Ez 1+5 SELECT. Az integrációs tesztben a fetch join változat ugyanazt az eredményt egy SELECT-tel adja. Az alkalmas javítást a konkrét lekérdezés alapján válaszd: fetch join, entity graph, batch fetch vagy célzott DTO-lekérdezés lehet indokolt.

Az ismétlődés önmagában nem bizonyít hibát: szándékos ismételt lekérdezés és polling is lehet. A küszöb csak diagnosztikai jelzés, nem automatikus kódmódosítás. Lapozásnál és több kollekciónál a fetch join más problémát is okozhat; a javítás eredményét és lekérdezésszámát is ellenőrizd.

Javítás előtti és utáni ellenőrzés

Válaszd ki a hibás kérés Root-ját, és nyomd meg a **Pin SQL baseline** gombot. Javítsd és fordítsd a kódot; támogatott módosításnál HotSwap is használható. Indíts új felvételt, hajtsd végre ugyanazt a kérést, majd válaszd ki annak Root-ját.

Az összehasonlítás megmutatja a SQL-darabszám, összes JDBC-idő és legnagyobb ismétlődés különbségét. A datasource, bemenet, cache és környezet legyen összevethető. Egyetlen rövidebb futás nem teljesítménybizonyíték. A részleges felvételeket a panel külön jelöli.

CASE-korlátok és JUnit

A **Cases / Reload > Options** két új mezője: **Maximum SQL executions** és **Maximum SQL repetition**. Üresen nincs SQL-állítás; a 0 megengedett, például adatbázis-hozzáférés nélküli kód ellenőrzéséhez. Tartományuk 0-1000000.

Az első mező a JDBC execute/executeQuery/executeUpdate és batch végrehajtások számát korlátozza; hibás SQL is beleszámít. Egy executeBatch egy végrehajtásnak számít, a batch belső elemszámát nem állítjuk lekérdezésszámnak. A második mező rootonként, datasource-onként és SQL-sablononként számol; nem csak SELECT-re érvényes. A kapcsolatfelvétel egyik darabszámba sem tartozik.

A mérési ablak a Code és Result expression részekre terjed ki. A bemenet betöltése, Imports, Setup, Cleanup és JSON-összehasonlítás kívül marad. Küszöbtúllépéskor **FAILED** az eredmény. Hiányos SQL-bizonyíték mellett nem lehet sikeres felsőkorlát-ellenőrzés: **INCONCLUSIVE** vagy **ERROR** jelenik meg. Agent nélküli runtime-ban a SQL-ellenőrzés hibaüzenettel megáll.

A **JUnit ZIP** megőrzi a SQL-korlátokat. A csomag hordozható DataSource-proxyt és SQL-normalizálót tartalmaz; a teszt JUnit/Spring környezetben agent nélkül fut. A JDBC-hozzáférésnek Spring által kezelt DataSource beaneken át kell mennie. Közvetlen DriverManager, natív unwrap, aszinkron munka vagy konkrét pool-osztály injektálása esetén a generált teszt adaptálást igényel. Ezt az export README-je is leírja.

MCP-használat

A katalógus 82 eszközt tartalmaz; a három új SQL-eszköz a 25. fejezet táblázatában szerepel. Ugyanaz a **Share IDE recordings with MCP** jogosultság és eszközönkénti engedélyezés érvényes. Olvasás és összehasonlítás nem futtat új Java-kódot.

- `repl_recording_sql`: `events`, `statistics`, `nextOffset`. Egy SQL-szöveghez `sql-id`, `text-offset`, `text-limit` használható; a `nextTextOffset` az esemény sorában van. Szöveglimit alapból 512, legfeljebb 2048 karakter.
- `repl_recording_findings`: `findings`, root, darabszám, idő, forrás és példa SQL/szülő-azonosítók. Az SQL-szöveg rövidített; a teljeshez az SQL-eszközt használd.
- `repl_recording_pin`: a hívás értékeivel együtt annak SQL-ágát is megőrzi a kliensnek.
- `repl_recording_sql_compare`: `sqlBefore`, `sqlAfter`, `sqlDelta`. Pontosan egy `before` vagy `reference` és egy `after` kell. A normál `repl_recording_compare` is megadja ezeket a statisztikákat.
- `recording` és `view` védi a lapozást. SQL-változás is új view-t eredményez. `partial=true`, `pending` vagy `dropped` mellett ne állítsd, hogy az N+1 biztosan megszűnt.

Claude-nak adható feladat: „Olvasd el az aktuális felvétel SQL-jeit és N+1-gyanús csoportjait. Azonosítsd a kérés rootját és az ismétlődő SQL alkalmazásbeli hívóját. Pineld a híváságat; javítás és új felvétel után hasonlítsd össze ugyanazt a kérést. Ne indíts üzleti műveletet külön kérés nélkül.”

Határok és megőrzött adatok

Szinkron Spring MVC és JDBC/JPA támogatott. A HTTP-gyökér akkor jön létre, ha a kérés eléri valamelyik kiválasztott osztályt. Az utána, ugyanazon dispatcher-szálon lefutó SQL is ide kapcsolódik. A HTTP-metódus és útvonal rögzíthető; query string, header és body nincs a HTTP-gyökérben.

Az SQL string-/számliteráljait és megjegyzéseit a normalizáló eltávolítja. Paraméterérték, ResultSet-tartalom és DB-jelszó nincs begyűjtve; hibánál a kivétel osztálya szerepel. A felvétel más alkalmazásadatot és forrást tartalmazhat, tehát nem általánosan anonimizált.

Külön keret a **1000 JDBC-esemény / 1,5 millió kódolt karakter**, a **200 Java-hívás** és a **32 MiB értékelőnézet**. Korlátozott, hiányos vagy folyamatban lévő adatnál jelzés látszik. Késői attach előtt létrehozott PreparedStatement SQL-je hiányozhat.

42. Hibernate a rögzített hívásfolyamban

A 0.22-es kiadás a Hibernate 6.6 ORM-eseményeit is hozzákapcsolja a felvételhez. Láthatod, melyik entity vagy kapcsolat lazy betöltése indított SELECT-et, történt-e flush vagy írás, illetve kiszolgálta-e a cache a lekérdezést. Az alkalmazás meglévő StatementInspector és event listener beállításai megmaradnak. A plugin nem kapcsolja be és nem nullázza a globális Hibernate Statistics számlálókat.

Bekapcsolás és a folyamat követése

1. A frissített plugin telepítése után indítsd újra az IDE-t és a célalkalmazást is, hogy az új agent fusson.
2. **Tap / Trace > Recorded calls > New recording...**: válaszd ki a controller/service/repository osztályokat. A JDBC/SQL opció mellett legyen bekapcsolva a **Record Hibernate 6.6 entity / session events** jelölőnégyzet.
3. Hajtsd végre az alkalmazásban a vizsgált műveletet, majd állítsd meg a felvételt. A **Root** szűrővel válaszd ki a kérést.
4. Az ORM-node-ra kattintva megnyílik a **Hibernate** részletező. A fa és a timeline ugyanazokat a rögzített eseményeket használja; az ORM-művelethez tartozó JDBC-node a gyermeke.

Például: Service-hívás > `Order.customer` lazy betöltés > `select ... where id=?`. A Java-szülő, ORM-szülő, root, szál és session külön azonosítóként szerepel. A session token nem entity-azonosító és nem újrafelhasználható objektumhivatkozás.

Gombok és szűrők

Felirat / hely	Mire való?
Hibernate részletező fül	Entity-betöltések, kapcsolatok, flush, dirty check, cache és tranzakciós események a jelenlegi Root-szűrőben.
Entity / relationship / source	Keresés entity- és kapcsolatnévben, leírásban, forrásosztályban és metódusban. Filter vagy Enter alkalmazza.
All events / Findings / eseménytípus	Minden esemény, csak a figyelmeztetésekhez kapcsolódók, vagy egy konkrét eseménytípus.
Esemény kiválasztása	Session, forráshely, idő, hibafajta, figyelmeztetés magyarázata és kapcsolódó SQL-ek jelennek meg. A részletek görgethetők.
Open originating call	A rögzített Java-hívót választja ki, annak forrásával és elmentett bemenetével/eredményével. Nem futtat kódot.
Open entity mapping	Az entity aktuális forrását nyitja meg a projektben. A jelenlegi kód eltérhet a felvétel idején futótól. Hiányzó forrást külön jelez.
Pin Hibernate baseline	Megőrzi az aktuális Root-szűrő ORM-számlálóit összehasonlításhoz; új felvétel után is megmarad.
Clear baseline	Törli a Hibernate-referenciát. Az SQL baseline és a Java Pin reference külön referencia.
Save / Open recording	Az ORM-naplót az SQL-lel és Java-hívásokkal együtt tárolja. A korábbi fájlverziókat is megnyitja.

Milyen eseményeket látok?

• **ENTITY_LOAD / INSERT / UPDATE / DELETE**: entity-művelet. Update-nél a módosult property-k neve látható, értékük nem.

• **LAZY_ENTITY / COLLECTION_INIT / LAZY_ATTRIBUTE**: proxy, persistent collection vagy bytecode-enhanced mező inicializálása. A kapcsolat például `Order.customer`; ha nem állapítható meg egyértelműen, unknown vagy multiple associations jelzés látható.

• **QUERY**: normalizált HQL-lekérdezés; a tényleges adatbázis-végrehajtásokat a hozzárendelt SQL-ek mutatják.

• **FLUSH / AUTO_FLUSH / DIRTY_CHECK**: flush, automatikus flush ellenőrzése és dirty checking. Az AUTO_FLUSH részlete mutatja, valóban szükséges volt-e flush.

• **TRANSACTION_BEGIN / COMMIT / ROLLBACK**, valamint **TRANSACTION**: tranzakcióhatárok és befejezési eredmény, sessionhöz kötve.

• **CACHE_HIT / MISS / PUT**: L2 cache eseményei. **QUERY_CACHE_HIT / MISS / PUT**: query cache eseményei. A plugin nem kapcsolja be a cache-t, és nem változtatja meg a cache-provider konfigurációját.

Az entity-load darabszám nem SQL-darabszám: egy query több entityt tölthet be, és cache-találat is létrehozhat managed entityt. A lazy inicializálás sem feltétlenül jelent SQL-t. Az L1 persistence contexthez nem állítunk teljes hit/miss számlálót. A Hibernate-időtartam magában foglalhatja a belső JDBC-munkát; ezeket az átfedő időket ne add össze.

N+1 és válaszírás közbeni lazy betöltés

A **suspected-n-plus-one** figyelmeztetéshez ugyanazon rootban, kapcsolatnál és forráshelynél ismétlődő lazy inicializálások és tényleges SELECT-ek kellenek. A küszöb a New recording N+1 mezőjéből jön; alapból 5. Cache-ből kiszolgált, SQL nélküli lazy műveletek önmagukban nem adnak ilyen jelzést.

A **lazy-during-response** azt jelzi, hogy a lazy betöltés a szinkron Spring MVC visszatérési értékének kezelése vagy későbbi renderelése közben történt, például JSON-serializálásnál. Ez külön figyelmeztetés; nem azonos minden controller után futó munkával. A magyarázat és a konkrét SQL-azonosítók a részletezőben szerepelnek. Mindkettő vizsgálati jelzés, nem automatikus bizonyíték hibás fetch-stratégiára.

Hasonlítsd össze ugyanazt a kérést azonos bemenettel, profillal és cache-állapottal. A **Pin Hibernate baseline** entity/lazy/flush és cache-számlálókat mutat előtte/utána; az SQL baseline adja a tényleges JDBC-darabszámot. Egy fetch join javításnál például az 1+5 SELECT egyre csökkenhet, miközben az eredmény változatlan.

Inspector: a megtekintés nem tölt be lazy adatot

Az Inspector a Hibernate-proxyt, persistent collectiont és megfigyelt entityt felismeri. Mutatja az inicializáltságot és a megfigyelhető managed/attached/detached állapotot. Betöltetlen proxy vagy collection tartalmát nem nyitja ki, és nem hív rajta size/iterator vagy entity getter műveletet. Már inicializált értéknél a betöltött mezők vagy a collection háttértárolója böngészhető.

Enhanced entity-nél a még nem betöltött mező **Unfetched Hibernate attribute; not read** jelzést kap. A null alapértéket nem állítja tényleges betöltött adatnak. Nem megfigyelt vagy nem támogatott entity-nél az állapot ismeretlenként jelenhet meg; a felület nem talál ki managed állapotot. Kifejezett REPL-kóddal továbbra is kérhetsz getterhívást, amely viszont valódi lazy betöltést indíthat.

CASE-korlátok és JUnit-export

A **Cases / Reload > Options** négy opcionális mezője:

Mező	Mit korlátoz?
Maximum entity loads	ENTITY_LOAD események száma.
Maximum Hibernate flushes	FLUSH, valamint a required=true AUTO_FLUSH események száma.
Maximum lazy initializations	Proxy-, collection- és enhanced-attribute inicializálások együtt.
Maximum lazy loads during response handling	A fenti lazy műveletek közül a szinkron MVC-válaszkészítéshez tartozók.

Üres mező: nincs ilyen assertion. A 0 érvényes; a tartomány 0-1000000. A mérési ablak a Code és Result expression részekre terjed ki; az input betöltése, Imports, Setup, Cleanup és összehasonlítás kimarad. Túllépés: FAILED. Részleges adat: INCONCLUSIVE, ha már egy ismert túllépés nem bizonyít FAILED eredményt. Hiányzó vagy nem támogatott adapter esetén a mérés nem indul el sikeres nullás számlálóval.

A **JUnit ZIP** megőrzi ezeket a korlátokat, és tartalmazza a hozzájuk illő `runtime/sb-repl-agent.jar` fájlt. Az ORM-es teszt JVM-jéhez szükséges:

```
-javaagent:/abs/path/runtime/sb-repl-agent.jar=port=0
```

Mavennél a Surefire argLine-ba, Gradle-nél a Test feladat jvmArgs opciójába tedd; a meglévő opciókat, például JaCoCo-t őrizd meg. A build daemon beállítása önmagában nem elég. A teszthez Hibernate 6.6 és egy már megnyitott session szükséges, amit a JPA/Spring inicializálás rendszerint létrehoz; szükség esetén a CASE Setup-ban nyiss és zárj egy sessiont. Az export README-je tartalmazza a feltételeket. Az agent helyi, hitelesített fejlesztői endpointot is indít. IDE nem kell; a kizárólag SQL-korlátot használó export továbbra is agent nélkül fut.

MCP és korlátok

A 25. fejezet három új eszköze: `repl_recording_hibernate`, `repl_recording_hibernate_findings`, `repl_recording_hibernate_compare`. A katalógus összesen **82 eszköz**. A meglévő **Share IDE recordings with MCP**, tool allowlist, maszkolás és audit érvényes. Olvasás és összehasonlítás nem hajt végre új alkalmazáskódot. A pin az ORM- és SQL-ágat is megőrzi; a statisztikák hibernateBefore/After/Delta mezőkben hasonlíthatók össze.

A kipróbált adapter **Hibernate 6.6.29.Final**, Java 17/21 alatt. Más Hibernate-verzióra nem jelentünk teljes mérést. Szinkron session/MVC/JDBC és támogatott, kapcsolt munkaszálak figyelhetők meg; reactive/R2DBC, StatelessSession és DB-belső execution plan/lock adatok nem részei ennek az adapternek. A késői attach előtti eseményeket nem lehet visszanyerni.

A Hibernate-napló kerete önállóan **1000 esemény / 1,5 millió kódolt karakter**, legfeljebb 64 ORM-szülőszinttel. Omitted/dropped, pending, unavailable vagy partial jelzés mellett a számok nem igazolják a teljes futás felső korlátját. A felvétel a kiválasztott osztályokhoz tartozó megfigyelt útvonalat mutatja; nem teljes alkalmazásprofilozó.

43. Rögzített hívásból futtatható CASE

Helye: Tap / Trace > Recorded calls. A 0.23-as munkafolyamat befejezett hívásból készít szerkeszthető reprodukciót, a metódus újrafuttatása nélkül.

1. Válaszd a **New recording...** műveletet és az alkalmazás osztályait.
2. A valódi kérés elindítása előtt kapcsold be a **Capture replay DATA** opciót. Ez külön engedélyezendő, mert a snapshot-szerializálók gettereket hívhatnak, és munkát adnak az alkalmazás szálának.
3. Jelölj ki egy befejezett Java-hívást, majd **Create CASE from call...**, vagy jobb kattintás a gráf csomópontján.
4. Adj új CASE-nevet, és válaszd ki a Spring beant. Statikus metódushoz nem kell bean. Több lehetséges fogadó esetén választani kell.
5. A **Cases / Reload** fülön frissíts, majd töltsd be az új CASE-t. Futtatás előtt nézd át a Code, Result expression, típus, elvárt kimenetel, Observed classes és SQL/ORM-korlátok mezőit.

Mi jön létre?

A művelet `név-input`, `név-expected` és `név` néven ment. A bemenet arg0, arg1 és további paramétereit belépéskor, a kimenetelt kilépéskor rögzíti. A későbbi objektummódosítások nem írják át ezeket a másolatokat. A generált Java az argumentumokat a deklarált publikus típusra alakítja, és a kiválasztott beant hívja, annak proxyviselkedésével együtt. JDK-proxyhoz publikus interfészt, CGLIB-proxyhoz az alkalmazás típusát használja.

Kivételnél típus- és üzenetelvárás készül. Teljes, elérhető SQL- és Hibernate-megfigyelés alapján induló korlátok is bekerülhetnek. Hiányos aszinkron vagy korlátozott felvételből nem készül bizonyítottnak tekintett felső korlát. Ezek áttekintendő kiindulási értékek, nem automatikusan elfogadott viselkedés.

Korlátok

A teljes adatfelvétel külön kerete **2 MiB bemenetenként/eredményenként, 32 MiB felvételenként**. Ez elkülönül az előnézetek és a normál DATA-tárolás keretétől. Publikus, elérhető metódustípusok és szerializálható adatok szükségesek. Future/CompletionStage helyett a munkaszálon elkészült értéket kell rögzíteni. Framework-infrastruktúra, nem támogatott típus és méretkorlát esetén látható hiba jelzi, hogy nincs megbízható replay.

A teljes replay DATA az aktuális runtime-felvételben él. A hordozható `.sbrepl-recording` megjelenítési bizonyítékot tárol, ezt a teljes DATA-t nem. A CASE-t készítsd el a felvétel lecserélése, Reset vagy kapcsolatbontás előtt. Az így létrejött tartós DATA és CASE a meglévő reproduction/workspace/JUnit funkciókkal exportálható.

A generált konverzió Jackson ObjectMappert és az elérhető modulokat használja. Alkalmazásspecifikus mixin, codec, security/tenant context és időfüggés külön áttekintést vagy Setup kódot igényelhet. A funkció reprodukciót készít elő; nem állítja vissza az egész alkalmazás futási környezetét.

44. DATA-másolat szerkesztése és variációk

Helye: Snapshots > Saved > **Edit DATA copy**. Előtte jelölj ki egy DATA snapshotot. Az Inspector **Edit DATA copy...** gombja először új eredeti DATA-ként rögzíti a kiválasztott élő értéket, majd ugyanezt a szerkesztőt nyitja meg.

Vezérlő	Működés
New DATA name	Eltérő, még nem használt név kell. A forrás DATA nem íródik felül.
Restore type	Az ellenőrzéshez használt deklarált Java-típus.
JSON editor	Formázott, leválasztott payload. Szerkesztése nem módosít élő objektumot.
Validate type	JSON-feldolgozás és típusos visszatöltési próba. Konstruktor és egyedi deszerializáló alkalmazáskódot futtathat.
OK	Újra ellenőriz, majd új DATA-másolatot ment. A forrás checksumának megváltozása elutasítja az elavult szerkesztést.
Cancel	Bezárja a szerkesztőt; a már elküldött mentést nem vonja vissza.

Az interaktív mérethatár **2 MiB**. Nagyobb DATA-ból előbb kisebb DTO-vetületet készíts, vagy exportáld. Az ellenőrzés a kiválasztott deklarált típusra vonatkozik; egy generikus Map nem minden belső üzleti DTO ellenőrzése. A másolat megőrzi a forrás nevét és verzióját.

Paramétervariációk

A **Cases / Reload** fülön jelölj ki egy mentett CASE-t, majd **Create variants...**. Adj új CASE-nevet és 1-20 különböző DATA-nevet, soronként egyet. Bemenetenként egy paramétersor készül. Kezdetben minden sor az eredeti expected DATA-t használja; a Parameters fülön nézd át és igazítsd az elvárásokat. A létrehozás nem futtat CASE-t és nem módosítja az eredeti definíciót.

Példa: rögzíts egy rendelést, készíts 0, 1 és 100 mennyiséget tartalmazó másolatot, majd háromsoros CASE-t. A valódi Spring beanek meghívása előtt határozd meg soronként az elvárt eredményt vagy kivételt.

45. Bean Explorer

Helye: az MCP utáni **Beans** fő fül. A **Java REPL > Tools > Bean explorer** is megnyitja. Az Insert bean továbbra is külön, gyors deklarációs segítség.

Vezérlő	Mire való?
Name / type és Search / refresh	Keresés név/típus alapján, a lazy beanek példányosítása nélkül. Enterrel is indítható.
Previous / Next	Lapozás, oldalanként 100 bean.
Bean kijelölése	Scope, primary/lazy, qualifier, alias, factory/forrás és elérhető proxy/target metaadatok.
Dependencies	Már feloldott függőségek és függő beanek. Dupla kattintással követhetők. Még nem inicializált kapcsolatok hiányozhatnak.
Metódusválasztó	Támogatott publikus metódus kiválasztása pontos JVM-szignatúrával, overloadokkal együtt.
Prepare method call...	Argumentumonként kompatibilis DATA választása és Java-kód beszúrása a munkafüzetbe. Nem hívja meg a beant és nem tölti vissza a DATA-t.

A DATA-javaslatok a deklarált típuson és óvatos raw-type illeszthetőségen alapulnak. Ezek jelöltek, nem minden mező sikeres konverziójának bizonyítékai. Ha nincs találat, előbb készíts vagy szerkessz a megfelelő típusú DATA-t. A biztonságosan meg nem nevezhető generikus szignatúrák nem választhatók.

Nézd át a generált kódot, használd a Check code funkciót, majd külön futtasd. Ez az utolsó lépés már deszerializál, lazy beant példányosíthat és valódi mellékhatást indíthat. A proxy-metaadat megfigyelés; az Explorer nem cseréli le és nem kerüli meg a Spring proxyt.

46. Watch és mezőváltozások

Helye: Java REPL eredményterület > **Watches**. A **Watch result** gomb `last1` alapértékkel nyitja a rögzítést; Pin watch alatt más sessionváltozó is megadható.

Vezérlő / nézet	Jelentés
Pin watch...	Változó/mező/index/kulcs útvonal: például last1.items[0].total vagy last1.items.size(). A hozzáadás még nem mintavételez.
Allow Java expression / method calls	Külön engedély tetszőleges Java-kifejezésre. Ez minden REPL-értékelés után alkalmazáskódot futtathat. Alapból kikapcsolt.
Refresh values	Kifejezett mintavétel az aktuális futtatási beállításokkal. Nincs watchokat értékelő időzítő.
Remove	A kijelölt watch eltávolítása ebből a sessionből.
Current / Previous	Az utolsó két leválasztott megjelenítési másolat kinyitása.
Changed fields	A korlátozott mezőútvonalak és értékek összehasonlítása.

A mintavétel kifejezett REPL-futtatás után történik, és nem cseréli le a last1/last2 értékét vagy az eredmény handle-jét. Az alapútvonal mezőket és biztonságos JDK-konténerhozzáférést használ; alkalmazásgettereket nem hív, a betöltetlen Hibernate-kapcsolatokat nem inicializálja.

Állapotok: FIRST, CHANGED, UNCHANGED, PARTIAL és ERROR. A PARTIAL, a korlát és az olvashatatlan mező nem bizonyít egyenlőséget. Hiba után a korábbi sikeres érték ERROR jelzéssel maradhat látható; ez nem friss eredmény. Sessionönként legfeljebb 20 watch és korlátozott before/after előnézet él. Reset/kapcsolatbontás törli őket. Az IDE és az MCP watchai a külön sessionjükhöz tartoznak.

47. Aszinkron flow és tranzakcióhatárok

Helye: New recording... > **Link Executor / @Async / CompletableFuture tasks**. Az IDE párbeszédablakában alapból be van jelölve; MCP-indításnál külön `async="true"` szükséges. A képességhez indítsd újra a célalkalmazást a friss agenttel.

Az **Async handoff** csomópont összeköti a megfigyelt beküldőt a támogatott executoron futó munkával. A feladathatárt szaggatott él jelzi. Látható a queueWaitMs, beküldési idő, tasktípus, induláskori cancelled állapot, valamint a beküldő és végrehajtó szál aktív tranzakciójának jelzése. A munkaszál Java-gyermekére kattintva nyílik meg a forrás és az adat; a szintetikus határnak nincs alkalmazásforrása.

Az adapter ThreadPoolExecutor, ForkJoinPool, Spring ThreadPoolTaskExecutor és SimpleAsyncTaskExecutor végrehajtást figyel. A csomagolt agent tesztjei Java 17/21-en CompletableFuture-folytatásokat és Spring @Async hívást is lefednek. A kapcsolt munkaszál SQL/ORM-művelete ugyanahhoz a gyökérhez tartozik, a tényleges szálazonosító megőrzésével.

Csak a felvétel azonosítása kerül át. A REPL nem másol tranzakciót, security/tenant adatot, MDC-t vagy tetszőleges ThreadLocal értéket. A szülő ROLLBACK tranzakciója ezért nem garantálja a munkaszál DB-írásának visszagörgetését. A nem Async folytatás helyben is lefuthat; egyedi executor, virtual-thread executor, Reactor és távoli üzenetsor általános követése nincs megvalósítva.

Legfeljebb 1024 várakozó taskazonosság él, ötperces élettartammal. Azonos Runnable bizonytalan párhuzamos újrafelhasználása, lejárat, GC és túlcsordulás el nem kapcsolt/eldobott bizonyítékként jelenik meg. A várakozó munka hiányossá teszi a mérést; Stop után az el nem indult kapcsolatokat elengedi. A gráf nem talál ki hiányzó szülőt. A határ taskvégrehajtást jelent, nem minden Future garantált sikerét; a Future belsejében elnyelt kivételhez a Java-gyermek eredményét is nézd meg.

A 4-es felvételformátum tárolja az explicit szálváltást és az async availability/pending/drop metaadatokat. A korábbi fájlok továbbra is olvashatók. A rögzítés és a fájl megnyitása nem folytat egy megállított stacket.

48. HotSwap és érintett CASE-ek áttekintése

Helye: Cases / Reload > **Reload + affected CASEs...**. Előbb válaszd ki a teljes újratöltendő Java-forrást, ahogy a Reload + run selected használatánál.

A plugin futtatás nélkül feldolgozza a forrást, majd az osztályneveket összeveti a CASE-ek **Observed classes** mezőjével. Az áttekintő párbeszédablakban a megfigyelt egyezések alapból kijelöltek. Külön jelölést kap az ismeretlen lefedettség és a megfigyelt egyezés hiánya; ezek az esetek is lehetnek érintettek. Legfeljebb 20 CASE választható.

Először a kiválasztott forrás HotSwapja történik meg. A mentett CASE-ek csak sikeres frissítés után futnak. Nem támogatott szerkezeti módosítás, fordítási hiba és megszakítás leállítja a munkafolyamatot. A CASE-szerkesztő módosításait külön mentsd el; mentett definíciók futnak. Nincs minden fájlmentésre automatikusan újraindított üzleti kód.

Összehasonlítás az előző futással

A riport előtte/utána kimenetelt, eredménylenyomatot, kivételt, SQL/ORM-számlálókat és időtartamot mutat, ahol van bizonyíték. Az első futás sessionalapot készít. CASE, DATA-verzió, Spring-context vagy futtatási beállítás módosítása esetén a régi alap nem összehasonlítható. A HotSwap így egyébként változatlan feltételek mellett vizsgálható.

Hiányzó/részleges számláló UNKNOWN. Az eredménylenyomat kanonikus, leválasztott JSON-ra vonatkozik, nem élő objektumazonosságra. Egy időmérés diagnosztikai támpont, nem teljesítményromlás bizonyítéka. A PASSED/FAILED eredményt az assertionök és a kifejezett korlátok döntik el. Az alapok és részletes eredmények sessionhöz kötöttek; a legutóbbi 20 CASE-riport marad meg.

49. MCP: új eszközök, események és feladatok

A katalógus már **82 eszközös**. Az új munkafolyamatokat az alábbi 16 eszköz éri el. A meglévő allowlist és külön execution, snapshot-write, capture, CASE-run, HotSwap és recording-sharing beállítás továbbra is érvényes.

Eszköz	Művelet
repl_bean_search	Bean név/típus keresés, oldalanként 100 sor.
repl_bean_info	Definíció, proxy, függőségek és szignatúrák.
repl_bean_compatible_data	DATA-jelöltek egy metódusparaméterhez.
repl_bean_prepare	Csak Java-előkészítés; eval előtt áttekintendő.
repl_snapshot_edit_read	Leválasztott payload/típus/forráschecksum.
repl_snapshot_edit_validate	Típusos deszerializálási próba; execution kell.
repl_snapshot_edit_copy	Új névre ment; execution és snapshot writes kell.
repl_case_variants	Paraméterezett CASE másolása, futtatás nélkül.
repl_case_affected	Javasolt esetek megfigyelt osztályok alapján; nincs reload/run.
repl_watch_add	Sessionwatch; allow-java külön engedélyezendő.
repl_watch_list	Watchazonosítók és utolsó állapot.
repl_watch_get	Utolsó before/after másolat és eltérések.
repl_watch_remove	Sessionwatch eltávolítása.
repl_watch_refresh	Kifejezett mintavétel MCP-futtatási móddal.
repl_recording_case_info	Teljes DATA elérhetősége az élő IDE-felvételben.
repl_recording_case_create	Tartós DATA/CASE létrehozása a rögzített hívásból.

A `repl_recording_start` új string boolean mezői: `capture-data` és `async`, MCP esetén alapból mindkettő false. CASE-létrehozáshoz execution, snapshot writes és Share IDE recordings kell. A létrehozás nem hívja újra a metódust. A külön MCP-evaluator a mentett CASE-t később saját futtatási módjával és CASE-run jogosultságával futtathatja.

Eseményfeliratkozás

A standard MCP-resource címe **repl://session/events**. A kliens listázza/olvassa, majd resources/subscribe művelettel feliratkozik. Az azonos sessionnel hitelesített GET /mcp, Accept: text/event-stream kapcsolaton resources/updated jelzések és taskállapot-értesítések érkeznek. A metadata lekéréséhez a kliens újraolvassa a resource-ot. Java-kód és rögzített érték nem kerül broadcastba.

A runtime napló capture-befejezést, contextváltozást, futtatási/CASE-befejezést és rögzített hívásokat jelez. Engedélyezett recording sharing mellett a közös IDE-felvétel változása is frissítési jelzést ad. A szerver belül másodpercenként lekérdezi a metaadatot; az AI-nak nem kell folyamatosan toolokat pollolnia. A resources/unsubscribe leállítja az előfizetést. Kliensenként egy folyam van; rövid újracsatlakozásnál Last-Event-ID használható.

A runtime 128 eseményt, az SSE 256 értesítést tart meg. A hiány és a lejárt replay látható: olvasd újra a resource/task állapotot, majd régi event ID nélkül kapcsolódj. A session és az események helyiek, hitelesítettek és elkülönítettek. A feliratkozás önmagában nem tart életben korlátlanul egy tétlen sessiont.

Azonosítható hosszú futások

MCP 2025-11-25 esetén az eval, CASE run/batch, reload és watch refresh opcionális task mezőt fogad a tools/call kérésben. Az első válasz working taskazonosítót ad. Tasks/get vagy notifications/tasks/status mutatja az állapotot; tasks/result adja az eredeti toolválaszt. Tasks/list a kliens saját megőrzött feladatait mutatja. Tasks/cancel kooperatív megszakítást kér és végleg cancelled állapotra vált; nem vonja vissza a mellékhatást, és nem szabadítja fel azonnal a még futó evaluatort.

Sessionönként legfeljebb 20 task marad meg. A TTL korlátozott, tényleges értékét a szerver közli. Az eredmény memóriában él, session/szerver megszűnésével elvész. Transporthiba után a task hibariportja megmarad, de a session további futtatást nem enged. Bizonytalan műveletet ne küldj újra automatikusan. Taskot nem támogató klienssel a szokásos szinkron toolhívás használható; a tényleges támogatás az MCP-hosttól függ.

Az MCP tasks specifikáció ezt kísérleti protokollfunkcióként írja le. Az MCP resources és a Streamable HTTP szabályozza a feliratkozást és az átvitelt.